For your Adan Pradan multitenant general e-commerce marketplace, you have already implemented three major foundations:

1. Identity and authorization — authentication, roles, permissions, and access control.
2. Organization/vendor lifecycle — vendor onboarding and organization upgrade workflows, plus the vendor plugin portal.
3. General product commerce lifecycle — the end-to-end physical product journey through refund.

Your next goal should be to cover the remaining business journeys that make the platform production-ready: financial settlement, inventory consistency, exceptions, customer/vendor operations, platform administration, and digital or subscription-based offerings.

I would organize the remaining work into the following implementation roadmap, assuming your current stack remains FastAPI, Next.js, PostgreSQL, Keycloak, SpiceDB/ReBAC, and Temporal.

1. Complete the remaining e-commerce journeys

These are the highest-priority gaps to examine first.

1. Payment lifecycle and reconciliation

High priority

- Payment pending, success, failure, timeout and retry.
- Duplicate payment and duplicate webhook handling.
- Partial payments, partial refunds and payment reversals.
- Gateway settlement and reconciliation against internal orders.
- Payment disputes and chargebacks.
- Failed payment recovery and order-expiry handling.

2. Vendor settlement and payout

High priority

- Calculate vendor earnings after commission, discounts and refunds.
- Maintain a ledger of vendor credits, debits and adjustments.
- Hold funds until the configured settlement condition is met.
- Generate vendor settlement statements.
- Process payouts, payout failures and retries.
- Reconcile gateway receipts with vendor balances.
- Handle negative balances and recovery of amounts already paid out.

3. Inventory and fulfillment exceptions

- Concurrent purchases of the last available unit.
- Inventory reservation, release and expiry.

- Out-of-stock items after checkout.
- Multi-warehouse allocation and split shipments.
- Vendor rejects an order after payment.
- Partial fulfillment, damaged parcels and lost shipments.
- Shipment failure, delivery failure and return-to-origin (RTO).

4. Order exception and recovery workflows

- Customer cancellation before and after vendor acceptance.
- Vendor cancellation and platform-initiated cancellation.
- Partial order cancellation.
- Return request rejected or disputed.
- Replacement instead of refund.
- Refund pending, failed or manually escalated.
- Compensation, store credit and goodwill adjustments, if supported.

Important: If any of these cases are already implemented in your general product journey, treat them as test and hardening tasks rather than rebuilding them.

2. Implement the customer journeys

The customer portal should cover the entire experience before, during and after a purchase—not only checkout.

Journey	What to implement
Customer profile	Registration, profile, addresses, preferences and account deactivation
Product discovery	Search, categories, filters, sorting, variants and product comparison
Wishlist	Add/remove products, stock alerts and price alerts if needed
Promotions	Coupon validation, discount rules, minimum order values and usage limits
Checkout	Address selection, delivery options, taxes, shipping charges and final price calculation

Order tracking	Order timeline, shipment tracking, delivery confirmation and downloadable invoice
Returns and replacements	Eligibility checks, return reason, evidence upload, pickup and resolution tracking
Reviews and ratings	Verified-purchase reviews, moderation, vendor responses and abuse reporting
Customer support	Tickets, order-related complaints, escalation and resolution
Notifications	Email, SMS or in-app updates for payment, dispatch, delivery, refund and account events

Essential customer scenarios to test

- A customer purchases from two vendors in a single cart.
- One vendor accepts the order while another rejects it.
- One shipment is delivered and another is delayed.
- The customer requests a partial refund for one item.
- The customer cancels an order while payment confirmation is still pending.
- A customer tries to access another customer's order by changing the order ID.

The last scenario is particularly important for your multitenant authorization model.

3. Implement the vendor lifecycle beyond onboarding

Since you already have vendor onboarding, organization upgrades and the vendor plugin portal, focus next on operational features.

Vendor catalog management

Product creation, variants, images, attributes, pricing, bulk import, draft/published status, category mapping and product approval.

Inventory and warehouse management

Stock adjustments, reservations, warehouse transfers, low-stock alerts, stock history and inventory reconciliation.

Vendor order operations

Accept/reject order, prepare package, generate shipping label, dispatch, mark exceptions, handle returns and upload proof of fulfillment.

Vendor finance and analytics

Gross sales, net sales, commissions, refunds, pending settlements, completed payouts, outstanding disputes and downloadable reports.

Vendor team management

Invite employees, assign store-level roles, restrict access to selected warehouses, manage staff and revoke access.

A vendor portal can share UI components and design with the customer and admin portals, but its navigation, permissions and operational workflows should be role-specific.

4. Implement the platform admin and tenant admin journeys

Your marketplace needs two different administrative layers. They should not have identical powers.

Platform Admin

Controls the marketplace as a whole.

- Approve, suspend and reinstate vendors.
- Approve or reject products and categories.
- Manage commissions, platform fees and marketplace policies.
- Manage payment gateway configuration and settlement exceptions.
- Handle escalated disputes and fraud cases.
- View cross-tenant metrics with appropriately restricted access.
- Manage platform-level plugins, feature flags and service health.

Tenant / Organization Admin

Controls a particular organization or tenant.

- Manage tenant profile, stores and organization settings.
- Invite and deactivate users within the tenant.
- Assign authorized roles and manage store access.
- Configure tenant-specific branding, policies and supported features.
- View tenant-specific orders, reports and activity.
- Manage subscriptions or plan entitlements, where applicable.

Additional admin workflows

- Vendor application rejection and resubmission.
- Vendor suspension while open orders or settlements exist.
- Tenant downgrade or cancellation when resources are in use.
- Product takedown while the product has pending orders.
- Manual refund or payout requiring approval.
- Audit trail for sensitive administrative actions.
- Data export and tenant offboarding/deletion workflows.

For each workflow, define who can initiate it, who can approve it, what happens to existing orders and funds, and whether it is reversible.

5. Implement digital products, software and subscription journeys

This is especially important because your platform is intended to support more than physical goods. Your existing physical-product order workflow should not be forced to handle every product type.

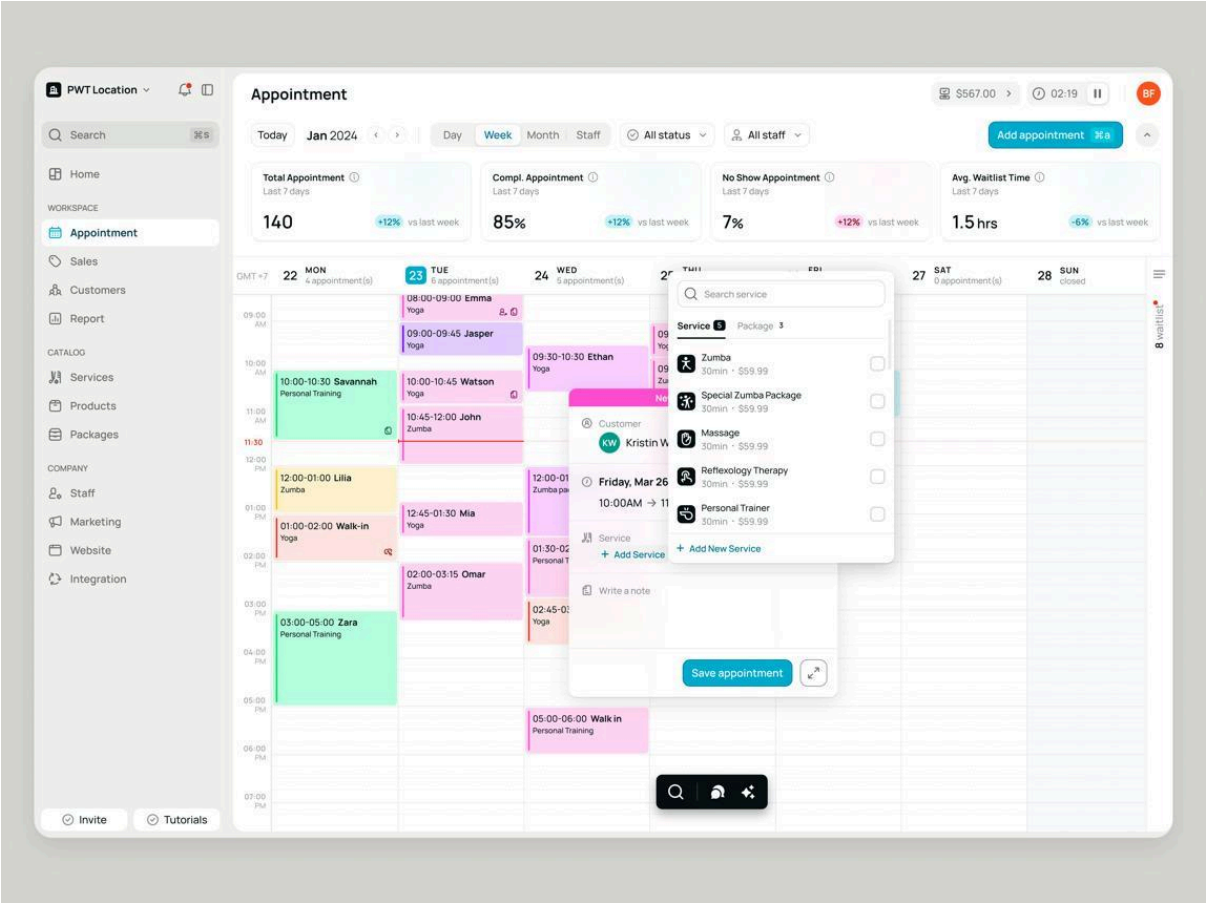
These are separate commerce journeys to build after the core marketplace is reliable.

A. Digital products and educational content

- Purchase a video course, e-book, notes or downloadable file.
- Grant access after confirmed payment.
- Enforce content permissions and download limits.
- Handle expiring links, revoked access and refunds.
- Track course access and download history.

B. SaaS subscriptions and software licensing

- Browse plans and compare features.
- Subscribe, upgrade, downgrade or cancel.
- Activate licenses and provision tenant resources.
- Enforce feature entitlements and usage limits.
- Handle renewals, failed recurring payments and grace periods.
- Suspend and reactivate access based on subscription state.



C. Services and appointment-based offerings

- Publish a service and its availability.
- Book a time slot and prevent double booking.
- Collect payment or a deposit.
- Reschedule, cancel, complete or mark a no-show.
- Handle service disputes and eligible refunds.

For your architecture, introduce a common Offering abstraction for purchasable items, with specialized fulfillment handlers for physical goods, digital goods, subscriptions and services. Keep their fulfillment lifecycles separate.

6. Cross-cutting journeys that make the platform production-ready

These may not appear as customer-facing pages, but they are essential workflows.

Journey	Important scenarios

Notifications	Retry failed email/SMS, avoid duplicate messages, manage delivery status
Audit and compliance	Record actor, tenant, action, resource, timestamp and outcome
Tenant isolation	Prevent cross-tenant data reads, writes, search results and exports
Background jobs	Retry transient failures, handle timeouts and escalate exhausted retries
Data migration	Schema upgrades, backfills, migration validation and rollback strategy
Observability	Correlation IDs, structured logs, metrics, traces and workflow failure alerts
Backup and recovery	Database backup, restoration testing and recovery procedures
Security	Rate limits, webhook signature validation, secret management and suspicious activity
Reporting	Tenant-level and platform-level sales, refunds, commissions and settlement reports
Data lifecycle	Retention, export, account deletion and legally required record preservation

Specific design rules for your stack

- PostgreSQL: Enforce tenant scoping in data access and use appropriate constraints and indexes. Consider row-level security as an additional defense where appropriate.

- Keycloak + SpiceDB: Authentication identifies the caller; authorization checks whether the caller can perform the requested action on the specific resource.
- Temporal: Use durable workflows for long-running operations such as payment recovery, vendor settlement, refund processing and subscription provisioning.
- Event handling: Use idempotency keys, transactional outbox patterns where appropriate, and deduplication for webhook and event processing.
- Financial records: Preserve an auditable ledger. Do not rely only on the current order status or vendor balance to reconstruct financial history.

7. Recommended implementation order

I would implement the remaining journeys in this sequence rather than developing all portals and features simultaneously.

Financial correctness

Payment edge cases, webhook idempotency, refund reconciliation, vendor commissions, settlement ledger and payouts.

Priority 1

2

Fulfillment exceptions

Multi-vendor orders, partial shipment, stock reservation, cancellations, delivery failures, returns and replacements.

3

Operational portals

Complete customer account and support features, vendor inventory and order management, and platform/tenant admin controls.

4

Production hardening

Tenant isolation tests, audit logs, retries, observability, migrations, backups, security checks and recovery drills.

5

Additional business models

Digital delivery, subscriptions, licenses, entitlement management, usage metering and service bookings.

8. Turn this into a trackable implementation checklist

Remaining journey tracker

0 of 25 additional workflows marked complete

Financial workflows
Payment failure, timeout and retry
Webhook idempotency and reconciliation
Vendor commission and settlement ledger
Payout processing and payout failures
Chargebacks and financial disputes

Fulfillment

Inventory reservation and concurrency
Multi-vendor and partial shipments
Cancellation edge cases
Delivery failure and return-to-origin
Replacement and partial refund

Portals

Customer profile, wishlist and reviews
Vendor catalog and inventory management
Vendor order fulfillment and finance dashboard
Platform admin and tenant admin workflows
Customer support and dispute escalation

Platform reliability

Tenant isolation and authorization tests
Audit trail and administrative approvals
Temporal retry, timeout and compensation tests
Outbox, event deduplication and notification retries
Backup, restore and deployment migration tests

New commerce models

Digital product delivery and access revocation
Subscription lifecycle and renewals
Entitlement and license provisioning
Usage metering and billing
Service booking and rescheduling

This checklist is a starting point, not a claim that all 25 workflows are missing. Mark a task complete only when its business logic, permissions, failure paths and tests are implemented.

My recommendation: Start with vendor settlement and financial reconciliation, followed by inventory/order exception handling. These are the most important gaps to validate after an end-to-end purchase and refund journey. Then complete portal operations and production hardening before expanding into subscriptions and digital products.

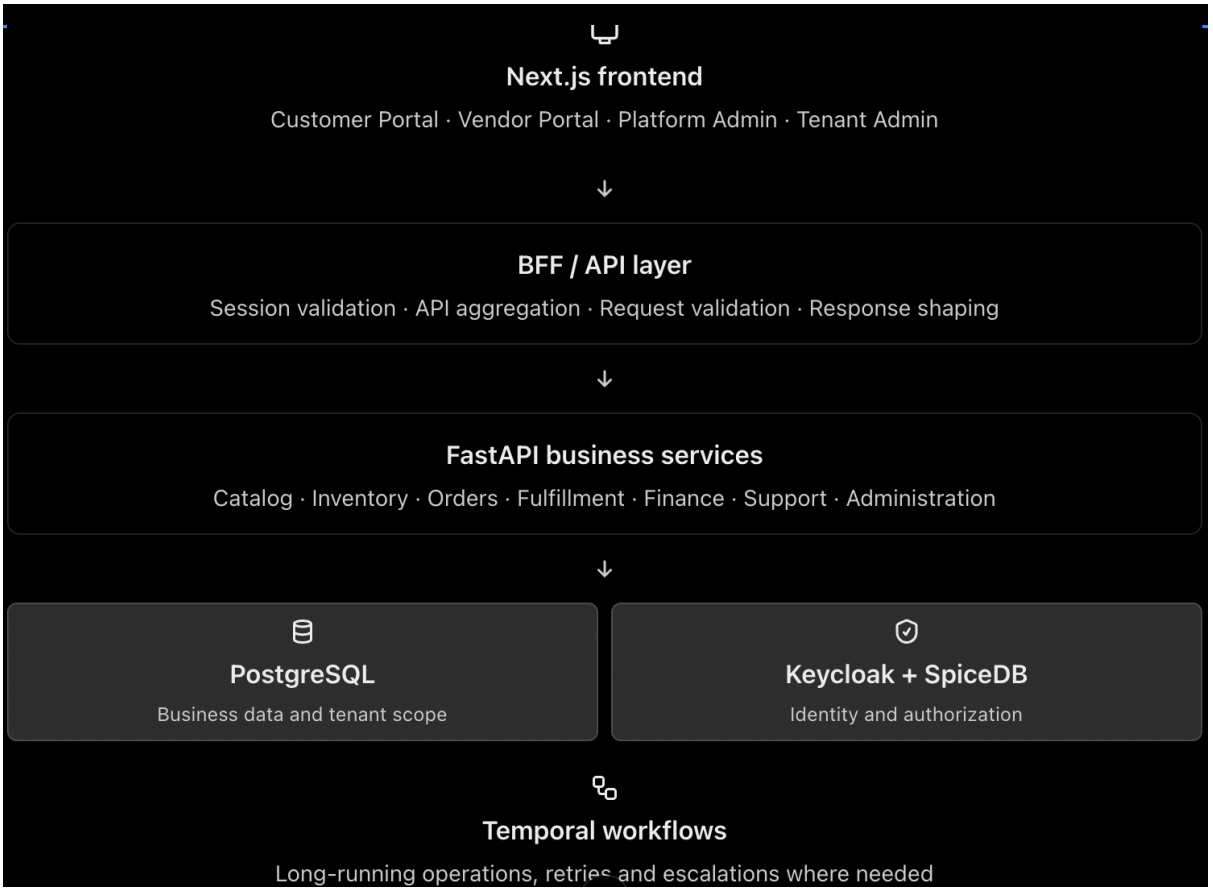
Since Journey 3 — Operational Portals is not implemented in your Adan Pradan multitenant e-commerce platform, I recommend implementing it as a separate, well-defined workstream without disturbing your existing Identity & Authorization, vendor/organization upgrade, vendor plugin portal, or product order-to-refund workflows.

The goal is to complete three operational experiences:

1. Customer portal — manage purchases, deliveries, returns, refunds and support.
2. Vendor portal — manage products, inventory, fulfillment and vendor finances.
3. Platform Admin + Tenant Admin portals — manage marketplace-wide operations versus individual organization operations.

The important architectural principle is that these are not simply three dashboards. They are role-specific interfaces over shared business services, with different permissions and workflows.

Step 1. Define the portal architecture



You do not need a separate backend or database for every portal. Reuse the services and schemas you already have wherever possible.

Step 2. Implement the portals in the right order

I recommend this order for your existing repositories,
and
.

1

Platform Admin + Tenant Admin

Start here

Establish operational controls, tenant boundaries and administrative workflows.

2

Vendor operations

Build catalog management, inventory, order processing, fulfillment and financial visibility around your existing vendor functionality.

3

Customer account and support

Build profile, address book, order history, tracking, return/refund status, support tickets and reviews.

Step 3. Design the required backend modules

First, inspect your existing modules and reuse the order, product, vendor and authorization logic already implemented. Do not create duplicate business logic just because a new portal needs a screen.

Backend domain	Main responsibility
	Product listing, variants, product details and publication
	Stock levels, adjustments, reservations and availability
	Order queries, cancellation eligibility and order history
	Vendor acceptance, packing, dispatch and delivery updates
	Commission breakdowns, settlements, payouts and financial statements
	Tickets, complaints, assignments and escalations
	Verified-purchase reviews, moderation and vendor responses

Vendor administration, policy configuration and operational exceptions

These are proposed domain boundaries, not a requirement to create eight new services or Python packages. If your existing modules already cover some of them, extend those modules instead.

These are illustrative contracts, not endpoints to add blindly. Confirm which routes already exist before implementing them. In particular, order acceptance and dispatch should invoke your existing order/fulfillment domain logic rather than directly updating database status fields from the route handler.

Step 4. Implement authorization before the UI

This is the most important step for a multitenant platform.

You already have Identity and Authorization implemented, so build on your existing abstraction and PostgreSQL-backed authorization model.

For every API operation, validate:

1. Identity: Who is making the request?
2. Role/permission: Can they perform the requested action?
3. Resource ownership: Does the product, order, store or ticket belong to the correct vendor, customer or tenant?
4. Lifecycle: Is the action valid in the current business state?

For example, a vendor must not be able to dispatch another vendor's order simply by supplying its order ID. A tenant admin must not access a different tenant's user records. A platform admin's cross-tenant access should be explicit and audited.

Avoid trusting a , or supplied by the browser as proof of ownership. Derive the caller's authorized scope from the authenticated identity and validate access to the requested resource.

Step 5. Build the Next.js portal interfaces

For the UI repository, create role-specific route groups using your existing application conventions.

Treat this as a proposed route structure, not a replacement for your existing app directory.

Build reusable components such as:

- Dashboard layout, sidebar and header.
- Paginated data table with filters.
- Status badge and lifecycle timeline.

- Confirmation dialog for destructive actions.
- Forms with validation and clear error handling.
- Empty states, loading states and retry controls.

Keep the underlying workflows distinct. For example, the customer order view and vendor order view can share a status timeline component, but they must expose different fields and permitted actions.

Step 6. Implement one complete vertical slice first

Rather than creating every page, build one feature all the way from database to browser.

I recommend starting with vendor product management, because it is a relatively contained way to establish the pattern for your other portals.

1. Inspect the existing product tables, SQLAlchemy models and Alembic migrations. Reuse them where they already exist.
2. Implement product list, detail, create and update use cases in the backend.
3. Add permission and vendor/tenant ownership checks to every relevant endpoint.
4. Add request/response schemas, validation, pagination and appropriate error responses.
5. Write backend tests for success, invalid data, unauthorized access and cross-tenant access.
6. Connect the Next.js product listing and editor to the API.
7. Test the entire journey through Docker Compose and your existing CI pipeline.

Once that pattern works, reuse it for inventory, fulfillment, admin operations and customer support.

Step 7. Define acceptance tests for each portal

Before marking a portal complete, verify its business behavior rather than merely confirming that the pages load.

Test	Expected result
Vendor creates a product	Product is associated with the authorized vendor and tenant
Vendor requests another vendor's product	Access is denied

Tenant admin requests another tenant's users	Access is denied
Customer requests another customer's order	Access is denied
Vendor dispatches an ineligible order	Business validation rejects the action
Platform admin suspends a vendor	Suspension follows defined rules for open orders and settlements
Customer submits a support ticket	Ticket is linked to the authenticated customer and relevant order
Unauthorized user accesses admin routes	API denies access regardless of UI visibility

Also test pagination, concurrency, duplicate requests, audit events and failures in any Temporal workflows triggered by these operations.

Your first implementation task

Start by auditing what is already present in both repositories. This avoids creating duplicate schemas, APIs or UI routes.

In , inspect:

- for existing domain modules, routers, services and models.
- for the authoritative schema history.
- Existing tests for identity, authorization, vendor lifecycle, orders and refunds.

In , inspect:

- for existing layouts, route groups and pages.
- Existing API clients, session handling and permission-aware navigation.
- Docker and CI configuration so new routes and tests fit your current deployment process.

My suggested next move: implement the vendor product-management vertical slice first, then use the same backend/API/UI/testing pattern for the remaining portal workflows. Once you have the current directory trees and existing API route list, I can map this plan to your actual modules and identify exactly what to create, what to reuse and what to test.

Adan Pradan: End-to-end marketplace implementation plan

Your initial objective is to launch a working multitenant e-commerce marketplace that you can operate through your own organization, Ninepoint Solution and Services LLP, before onboarding more organizations and vendors.

You have already completed Identity and Authorization, vendor and organization upgradation, and the vendor plugin portal. Your next phase is to implement the complete commerce lifecycle, including the customer marketplace, inventory, checkout, payments, shipment, billing, refunds, settlement and operational logging.

I recommend treating this as a commerce platform implementation, not simply a collection of portal pages.

1. Your initial business scenario

Platform owner organization

Ninepoint Solution and Services LLP

Initially, you and two organization members will operate the platform.

Initial vendor onboarding

Vendor A

Stationery and gifts

Physical inventory, packaging, shipment, delivery and returns.

Vendor B

Software solutions

Software licenses, digital delivery and potentially service offerings.

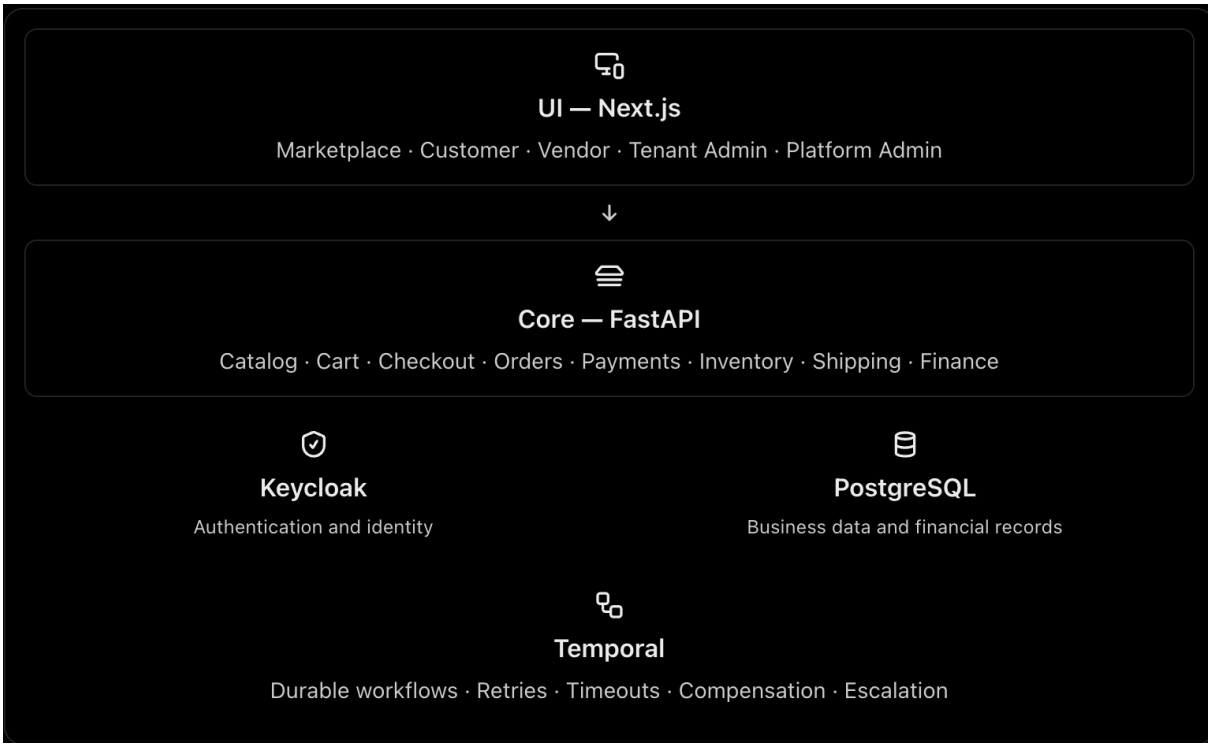
Customer marketplace

Customers discover products from both vendors, shortlist items, maintain a wishlist, add products to a cart, see the total payable amount, complete checkout and pay.

One important distinction: stationery and gifts follow a physical fulfillment lifecycle, while software solutions may require license activation, downloads, subscriptions or service delivery. Build a common checkout and payment foundation, but keep the fulfillment journeys separate.

2. Your existing technology architecture

I would preserve the architecture you have already established.



Keep the existing separation of responsibilities:

- Keycloak authenticates users and issues identity tokens.
- Core enforces authorization and executes business rules.
- PostgreSQL stores marketplace data, orders, inventory, payments, returns, invoices and financial ledgers.
- Temporal coordinates long-running operations that span multiple steps or external systems.
- UI presents role-specific experiences and invokes Core APIs.

Continue to keep Keycloak's own database separate from your application business schema and Temporal's persistence database. Do not store payment card details in your application database; use a payment gateway's hosted checkout or tokenized payment flow.

3. Theory plan: the journeys to implement

I would divide the implementation into 12 connected business journeys. Each journey should define the actor, business rules, data changes, workflow states, failure cases and audit events before you start coding.

Journey 1 — Organization, tenant and vendor setup

Goal: Make Ninepoint operational and onboard the first two vendors correctly.

1. Set up Ninepoint's organization profile, tenant configuration and authorized organization members.
2. Invite the two initial members and assign appropriate permissions.
3. Onboard the stationery/gift vendor and software-solutions vendor.
4. Configure each vendor's profile, business details, payout details, commission policy and supported product types.
5. Activate vendors only after the required approval and verification steps are complete.

Key design decision: Establish the precise ownership model before creating commerce records.

- Platform: operates the marketplace.
- Tenant/organization: the organization boundary used by your application.
- Vendor: owns its catalog, inventory, fulfillment operations and vendor financial records.
- Customer: buys products and owns their cart, wishlist, orders and support cases.

Whether each vendor is a separate tenant or an organization within a tenant must follow your existing organization model. Do not create a second, conflicting tenancy model just for commerce.

Journey 2 — Vendor catalog and product publishing

Goal: Allow vendors to add products that customers can discover and purchase.

Example: stationery vendor publishes a notebook

Draft
Validated
Approved
Published

Required product information:

- Name, description, category and images.
- SKU, product variant, price and currency.
- Tax classification and applicable tax rules.
- Physical or digital fulfillment type.
- Inventory tracking policy and availability.
- Shipping eligibility and return policy.

The vendor must be able to create, edit, deactivate and view products. The platform admin should be able to moderate products if marketplace policy requires approval.

For software products, additionally support license type, activation instructions, delivery method and any subscription or service terms that apply.

Important: Product price and inventory are separate concepts. Publishing a product does not guarantee that it is available for purchase.

Journey 3 — Customer marketplace, shortlist and wishlist

Goal: Let customers discover products from both vendors in one marketplace.

The customer should be able to:

- Browse categories and search across vendors.
- Filter and sort products by price, type, availability and other attributes.
- Open product details and compare relevant information.
- Shortlist products for comparison or later consideration.
- Add or remove products from a persistent wishlist.
- See product availability, estimated shipping information and return conditions.
- View vendor identity and product fulfillment type.

I recommend distinguishing three concepts:

Concept	Purpose
Shortlist	A temporary or named collection of products the customer is considering
Wishlist	Products saved for future purchases, potentially across sessions
Cart	Products currently intended for purchase, with selected quantities and variants

You may choose to make shortlist a feature of wishlist in your first release, rather than creating two separate systems.

A wishlist does not reserve inventory. Prices and stock can change between saving an item and checkout.

Journey 4 — Cart and pricing calculation

Goal: Calculate an accurate total before payment.

Suppose a customer selects:

- Notebooks from Vendor A.
- A gift item from Vendor A.
- A software license from Vendor B.

The cart should support quantity changes, item removal, variant changes, coupon validation and availability checks.

The total calculation should be explicit:

$$\text{Subtotal} = \sum_i (\text{unit price}_i \times \text{quantity}_i)$$

$$\text{Payable} = \text{Subtotal} - \text{Discounts} + \text{Tax} + \text{Shipping} + \text{Fees}$$

The exact tax calculation order and whether prices include tax depend on the applicable tax rules and jurisdiction.

The customer should see a clear breakdown of the subtotal, discounts, shipping, taxes and final payable amount. The server—not the browser—must calculate the authoritative amount.

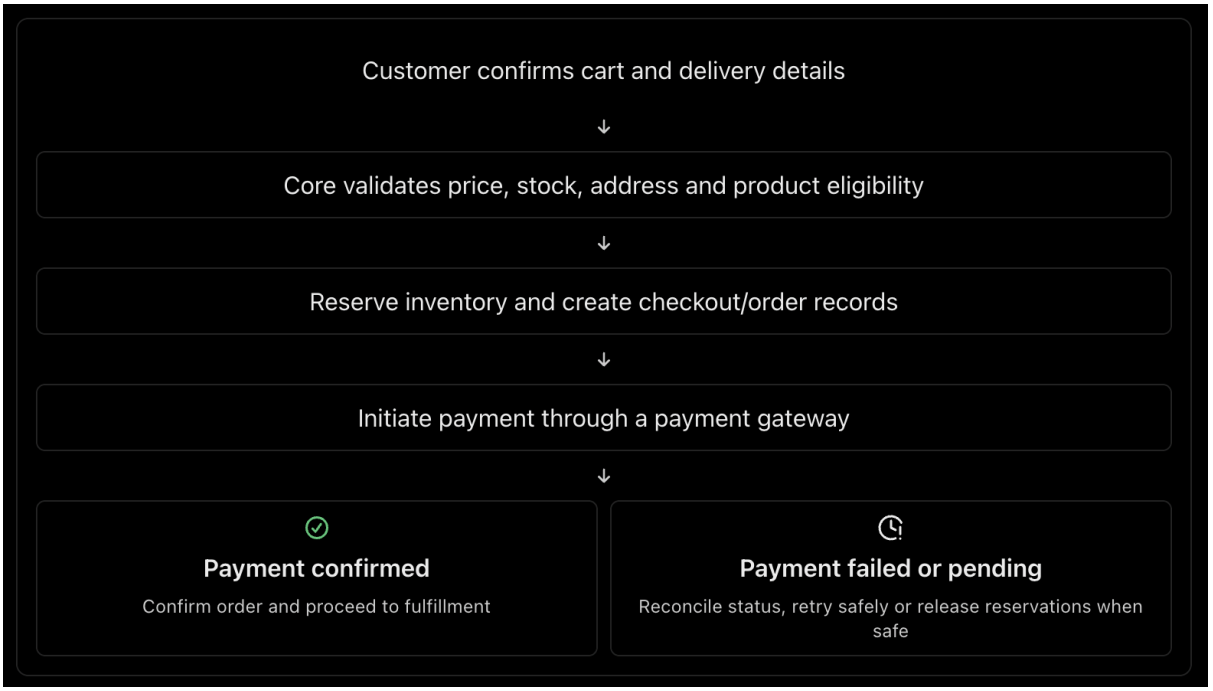
Critical scenarios:

- One item goes out of stock before checkout.
- A vendor changes the price after the item was added.
- A coupon expires.
- One vendor cannot fulfill its part of the cart.
- The customer submits checkout twice.

These must not result in incorrect totals, overselling or duplicate orders.

Journey 5 — Checkout, order creation and payment

Goal: Convert a cart into a valid, payable order.



Use a payment gateway appropriate for your target market. The integration should support payment initiation, signed webhook verification, payment status reconciliation and refunds.

Do not mark an order paid merely because the browser redirects to a success page. Confirm the payment through a trusted server-side notification or gateway status verification.

For a cart spanning multiple vendors, create a parent order and vendor-specific suborders. The parent represents the customer's overall purchase; each suborder supports the relevant vendor's fulfillment, returns and settlement.

For a first release, I recommend one payment for the cart, with allocations to the individual vendor suborders. Handle partial cancellations and refunds through explicit allocation records.

Journey 6 — Vendor inventory management

Goal: Maintain correct stock for each vendor and product.

Each inventory record should identify the product or variant, owning vendor, applicable tenant or organization, and—if supported—warehouse.

A useful inventory model distinguishes:

- On hand: physical units recorded in stock.
- Reserved: units allocated to active checkouts or orders.
- Available: units eligible for new purchases.
- Damaged or quarantined: units not available for sale.

A basic relationship is:

$$Available = Onhand - Reserved - Unavailablestock$$

Define the exact inventory accounting rules so units are not deducted twice when an order is placed, shipped or returned.

The vendor must be able to receive stock, adjust quantities with an audit reason, view stock movements and receive low-stock alerts.

The system must handle concurrent purchases of the last item safely. Use database transactions and appropriate row locking or atomic inventory updates; application-level checks alone are insufficient.

Journey 7 — Shipment, delivery and fulfillment

Goal: Deliver the correct physical products to customers.

Physical product lifecycle

Order accepted
Packed
Dispatched
In transit
Delivered

Essential functionality:

- Vendor acceptance or rejection.
- Shipment creation and shipping-label reference.
- Courier, tracking number and tracking updates.
- Partial shipments and multiple packages.
- Delivery confirmation and failed delivery handling.
- Lost, damaged and return-to-origin shipments.
- Shipping charges, delivery evidence and customer notifications.

Start with manual shipment creation and tracking updates if necessary, but design a shipping-provider integration boundary so courier APIs can be added later.

Software licenses should instead follow a digital fulfillment lifecycle such as payment confirmed → entitlement provisioned → customer notified → activation confirmed. They should not require physical shipment.

Journey 8 — Cancellation, return, replacement and refund

Goal: Resolve post-purchase problems without corrupting order, payment or inventory records.

Keep these as related but distinct processes:

- Cancellation: stop an order or an unfulfilled item.
- Return: accept a request to send an already delivered physical item back.
- Replacement: fulfill a replacement item, potentially without a refund.
- Refund: return some or all of the money to the customer.

The return lifecycle might be:

A refund has its own lifecycle:

For partial refunds, track which order items and amounts are affected. For returned physical goods, record the disposition of the item and update sellable inventory only after the appropriate inspection and restocking decision.

For digital products, define separate refund and entitlement-revocation policies. Revoking access must not happen automatically in every case without considering the applicable business rules.

Journey 9 — Customer billing, vendor billing and platform billing

Goal: Make every amount traceable to a business event.

These are three distinct financial views, and they should not be implemented as three unrelated calculators.

Customer billing

- Order confirmation and final price breakdown.
- Customer invoice or receipt as legally appropriate.
- Credit notes or adjustment documents where required.
- Payment and refund history.

Vendor financial statement

- Vendor sales and fulfilled orders.
- Commission and applicable fees.
- Tax withholding or other adjustments where legally applicable.
- Returns, refunds and chargebacks.
- Pending balance and payout history.

Platform financial reporting

- Gross merchandise value and collected payments.
- Platform commissions and other earned fees.
- Payment gateway fees and operating expenses.
- Refund liabilities and adjustments.
- Vendor payables, payouts and reconciliation differences.

A crucial distinction: customer payment collected is not the same as platform revenue. The platform's revenue depends on its contractual arrangement, commission rules and applicable accounting treatment.

Keep invoices, payment transactions, vendor statements and the financial ledger as separate but linked records. Determine invoice numbering, tax documents and statutory requirements with a qualified accountant for the jurisdictions in which you operate.

Journey 10 — Vendor settlement and payout

Goal: Calculate how much each vendor is owed and pay it accurately.

A typical sequence is:

1. Record the confirmed customer payment.
2. Allocate the payment across the parent order and vendor suborders.
3. Apply the commission and other agreed adjustments.
4. Track delivery, return-window and dispute-related holds according to policy.
5. Create the vendor's payable balance.
6. Generate a settlement statement.
7. Initiate the payout using the configured payment provider or bank process.
8. Reconcile the payout result and record failures or reversals.

Use a financial ledger with auditable debit and credit entries. Do not derive the entire financial history solely from mutable order statuses.

For your first two vendors, payouts may be manually approved and initiated. The data model should nevertheless support payout references, status, amount, currency, settlement period and reconciliation.

Do not assume a payment gateway automatically supports marketplace split payments or vendor payouts. Verify the provider's actual capabilities, onboarding requirements and country-specific rules before selecting the integration.

Journey 11 — Vendor plugin portal operations

Goal: Make your existing vendor plugin portal useful for daily operations.

The plugin portal should be the vendor's operational workspace, not just a plugin installation interface.

Capability	Minimum functionality
Dashboard	Orders awaiting action, low stock, sales and outstanding settlements
Product management	Create, edit, publish and deactivate products
Inventory	Adjust stock, review movements and handle availability
Orders	Accept/reject, pack, dispatch and view order details
Returns	Review return requests and record inspection outcomes
Finance	Sales breakdown, commission, payable balance and payout history
Plugin management	View enabled plugins, configuration, permissions and health
Team access	Manage authorized vendor staff and their capabilities
Notifications	Receive actionable order, inventory and payout alerts

For plugins, add a controlled lifecycle: installation or enablement, configuration validation, permission checks, execution, error reporting, disablement and audit history.

A plugin must not bypass Core authorization, mutate another vendor's data, or independently change financial records without going through the relevant business service.

Journey 12 — Logs, audit trail and operational recovery

Goal: Make the marketplace diagnosable and safe to operate.

You need several complementary mechanisms.

Mechanism	What it answers
Application logs	What happened during a request or business operation?
Audit trail	Who performed a sensitive action, on which resource, and when?
Temporal workflow history	Which workflow activities ran, retried or failed?
Business event history	How did an order, payment, return or settlement change state?
Metrics and alerts	Are errors, payment failures, stuck orders or queue delays increasing?

For every relevant event, capture identifiers such as , , , , and where applicable.

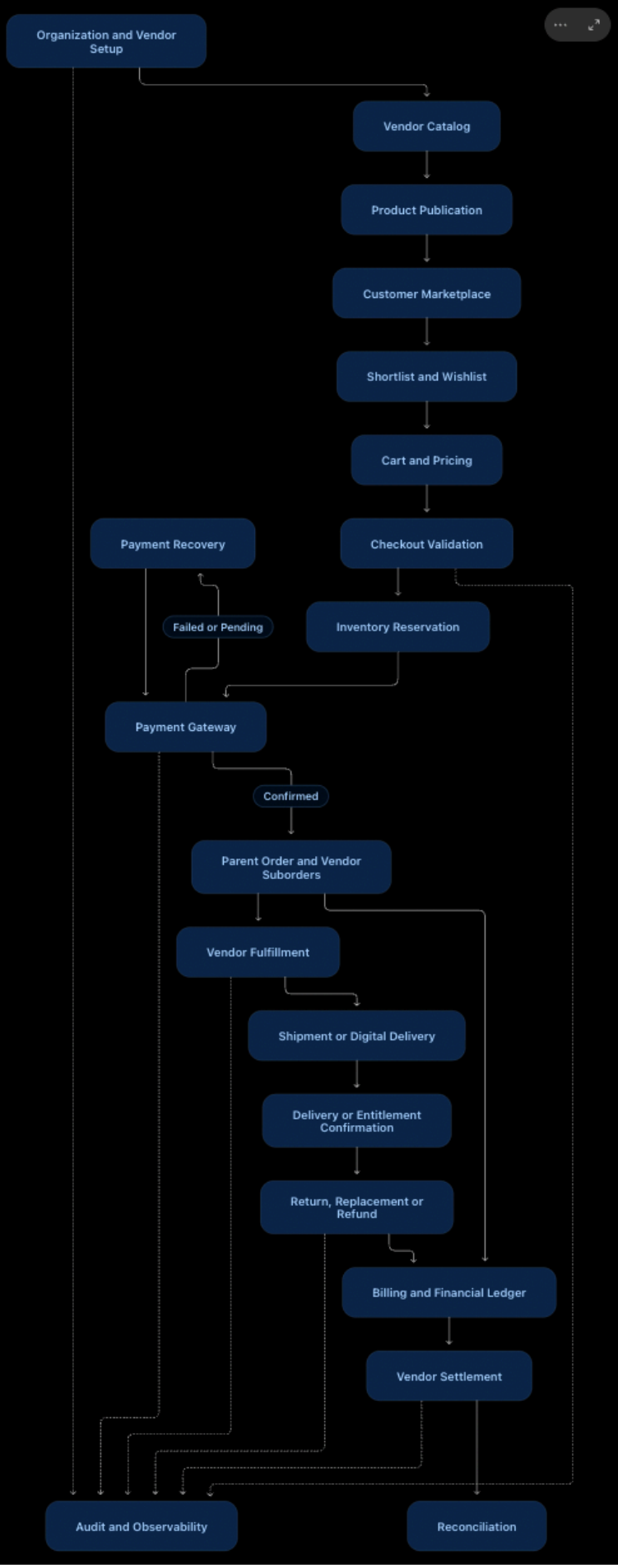
Never log passwords, access tokens, session secrets, payment card data or other sensitive credentials. Restrict audit access and define retention policies.

Also include notification delivery, customer support, product moderation, disputes and data export/deletion policies in your wider operational plan. These are easy to overlook when focusing only on the purchase journey.

4. How the journeys connect end to end

The most important architectural decision is to avoid implementing each journey as an isolated feature.

The most important architectural decision is to avoid implementing each journey as an isolated feature.



A single purchase may trigger several independent lifecycles:

- An order lifecycle tracks commercial fulfillment.
- A payment lifecycle tracks money collected from the customer.
- An inventory lifecycle tracks stock and reservations.
- A shipment or digital fulfillment lifecycle tracks delivery.
- A return/refund lifecycle tracks post-purchase resolution.
- A settlement lifecycle tracks money owed and paid to vendors.

These lifecycles must be related, but they should not be collapsed into one field. For example, an order may be delivered while its vendor payout is still pending.

5. Where Temporal should be used

Use Temporal when an operation must survive service restarts, wait for an external event, retry an operation or coordinate multiple business steps. Keep simple, short database operations in Core's normal request/transaction flow.

Workflow	Recommended approach
Checkout and payment recovery	Temporal coordinates reservation, payment confirmation, timeout and compensation
Vendor order processing	Use a workflow if fulfillment requires durable multi-step coordination
Shipment tracking	Process incoming carrier events idempotently; use workflows for time-based escalation where useful
Return and refund	Coordinate approval, pickup/inspection, refund and stock disposition where applicable
Vendor settlement	Coordinate calculation, approval, payout and reconciliation
Notifications	Use retryable asynchronous delivery with deduplication

Product CRUD and wishlist updates	Normal Core API and database transactions are usually sufficient

A critical payment design rule

Temporal does not make an external payment operation exactly-once by itself. A workflow activity can be retried after a timeout even if the payment provider already processed the request.

Therefore:

- Use stable idempotency keys for payment and refund operations.
- Persist payment attempts and gateway references.
- Verify webhook signatures and deduplicate incoming events.
- Reconcile ambiguous payment states before releasing stock or retrying a charge.
- Use compensation and reconciliation rather than assuming a database rollback can undo an external payment.

For a multi-vendor order, define what happens if the customer pays successfully but one vendor cannot fulfill its suborder. The platform must apply a documented cancellation, partial-fulfillment or refund policy.

6. Recommended implementation phases

Implement in dependency order. Complete and test one phase before adding the next major workflow.

1

Foundation and vendor catalog

Organization configuration, initial vendor setup, product categories, product variants, publishing, pricing and inventory schema.

Deliverable: both vendors can publish purchasable products.

2

Marketplace, cart and checkout

Search, product details, shortlist, wishlist, cart, pricing breakdown, addresses, shipping options and order preview.

Deliverable: customer can prepare a valid order and see the exact payable amount.

3

Inventory, checkout and payment

Atomic inventory reservation, order creation, payment gateway integration, webhook processing, timeout recovery and duplicate-request handling.

Deliverable: a customer can complete a purchase reliably.

4

Vendor fulfillment and delivery

Vendor order management, shipment tracking, digital product delivery, order cancellation and delivery confirmation.

Deliverable: paid orders can be fulfilled through the correct product-specific workflow.

5

Returns, refunds and financial operations

Customer billing, vendor statements, platform accounting, commission calculation, settlement, payout and reconciliation.

Deliverable: every payment, adjustment, refund and vendor payable is traceable.

6

Operational readiness

Customer support, admin dashboards, audit logs, metrics, alerts, security testing, backups and end-to-end CI/CD tests.

Deliverable: Ninepoint can operate the marketplace and investigate failures.

7. Essential data model to plan before coding

Your existing identity, organization and vendor schemas should remain the source of truth for those domains. Add or extend the following commerce entities according to what is already present.

Domain	Main entities
Catalog	, , ,
Inventory	, ,
Customer shopping	, , , ,
Checkout	,
Orders	, , ,
Payment	, , ,
Delivery	, , ,
Returns	, ,
Billing	, ,
Vendor finance	, , ,

Operations , , ,

This is a conceptual inventory, not a requirement to create every table immediately. Some entities may already exist or belong in separate modules. Decide which data is authoritative, how records relate to tenant/vendor ownership, and which records must be immutable before defining migrations.

8. Define your initial launch scope

For the first working release, I would keep the scope deliberately narrow.

Initial release checklist

0/15

Organization and access

Ninepoint organization configured
Two organization members invited with appropriate roles
Two vendors onboarded and authorized

Shopping

Physical products and software listings
Search, product details, shortlist and wishlist
Cart, quantity updates and authoritative price calculation
Checkout and customer payment

Operations

Vendor inventory and order management
Physical shipment tracking and digital delivery
Cancellation, returns and refund processing
Customer invoice and vendor financial statement

Platform control

Platform commission and vendor settlement
Platform billing and reconciliation
Audit trail, error logs and workflow monitoring
Tenant isolation and end-to-end automated tests

Do not postpone failure handling until after the happy path works. Payment ambiguity, duplicate webhooks, overselling, vendor rejection, failed delivery and refund reconciliation need to be considered during the relevant phase.

9. What I recommend doing next

Before writing code, create a journey-to-module mapping against your current Core and UI repositories. For each journey, record:

- What already exists.
- What must be extended.
- What is genuinely missing.
- Which database migrations are required.
- Which API contracts and authorization rules are needed.
- Whether Temporal is required.
- Which success, failure and recovery tests prove completion.

This will prevent duplicate modules and help you preserve the Identity and Authorization, organization/vendor upgrade, and plugin functionality you have already built.

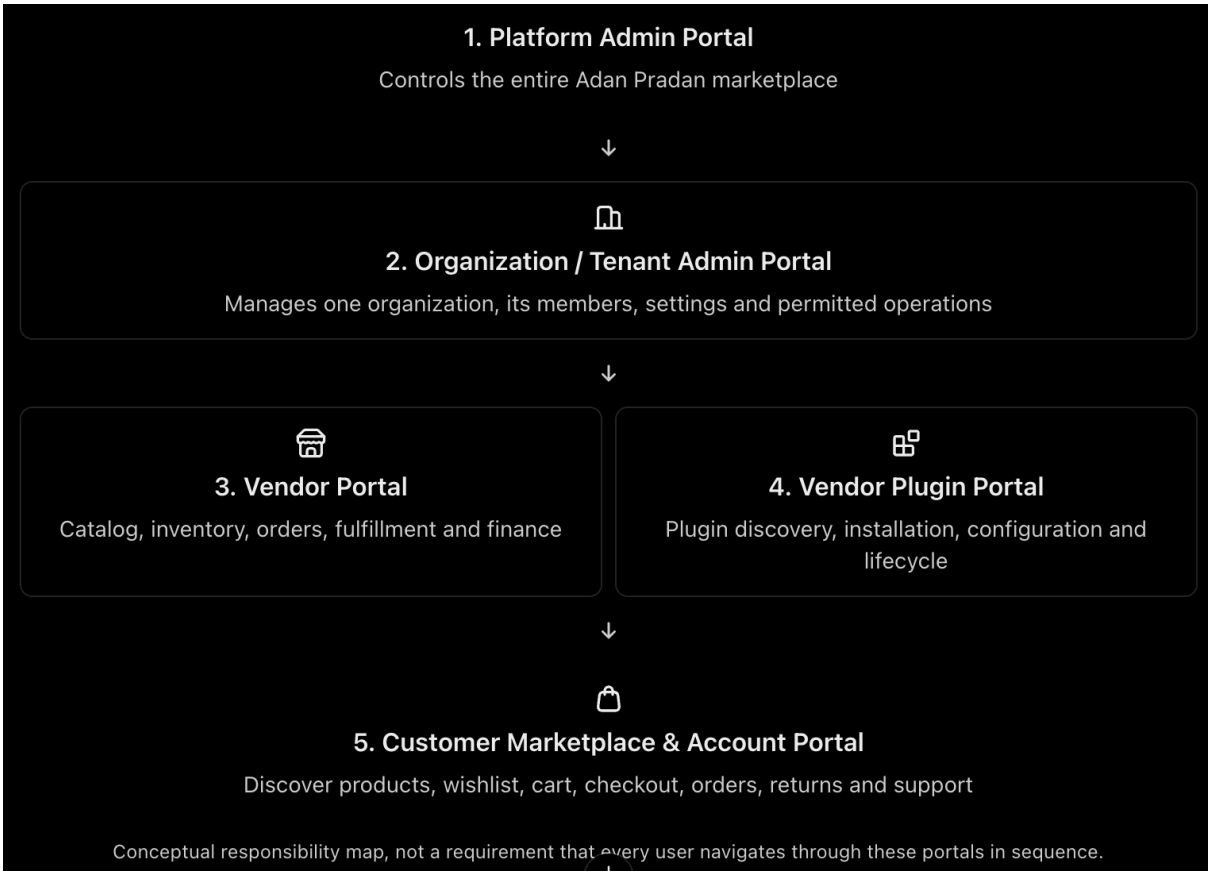
The best next step is to start with Phase 1: vendor catalog and inventory foundations, after auditing your existing product, vendor, organization and migration schemas. Once those foundations are clear, the marketplace, cart, checkout and payment workflows can be implemented on a stable data model.

For your Adan Pradan multitenant e-commerce platform, I recommend designing 5 distinct portal experiences, with one additional optional portal for delivery partners later.

The important distinction is that a portal is a separate user experience, navigation and set of permissions. It does not necessarily mean a separate Next.js application, backend or database.

You already have Identity and Authorization, organization/vendor upgrade workflows, and a vendor plugin portal. Your UI should build on these rather than duplicate them.

1. Recommended portal structure



One nuance: the vendor portal and vendor plugin portal are separate experiences, but they can share the same vendor layout, authentication and navigation shell. If your existing plugin portal is already a section of the vendor workspace, keep it that way instead of building a second application.

2. What each portal should contain

Portal 1 — Platform Admin

For you as the marketplace operator

- Marketplace overview and KPIs.
- Organization and vendor approvals, suspension and upgrades.
- Product moderation, category and marketplace policy management.
- Orders, escalated refunds, disputes and settlement monitoring.
- Commission rules, platform financial reporting and reconciliation.
- Audit logs, plugin governance and system health.

Portal 2 — Organization / Tenant Admin

For the organization owner and authorized organization members

- Organization profile and configuration.
- Invite, deactivate and manage members.
- Assign roles and permissions within authorized scope.

- Manage stores, vendor relationships and organization-level settings where applicable.
- View organization-scoped reports and activity.

Portal 3 — Vendor

For your stationery/gifts vendor and software-solutions vendor

- Product catalog, prices, variants and publishing.
- Vendor-specific inventory and stock movements.
- Incoming orders, packing, dispatch and delivery.
- Cancellations, returns and refund processing.
- Sales, commission, invoices/statements and settlements.
- Vendor team, notifications, reports and settings.

Portal 4 — Vendor Plugin Management

For vendors who configure extensions to their workspace

- Browse available plugins.
- Install, enable, disable or uninstall authorized plugins.
- Configure settings and credentials securely.
- View compatibility, permissions, health and execution errors.
- Track versions, updates and plugin audit events.

Portal 5 — Customer Marketplace & Account

For shoppers, including guest visitors if you choose to support them

- Product discovery, search, filters and product details.
- Shortlist, wishlist and shopping cart.
- Address management, price breakdown and checkout.
- Payment, order history and shipment tracking.
- Cancellation, returns, refunds, reviews and customer support.

3. Do you need more portals?

Not immediately. These are the additional experiences I would plan for, without building them as separate portals in the first release.

Experience	Recommendation
Delivery partner / logistics portal	Add later if your own courier partners need to manage pickups, shipments and proof of delivery.

Customer support agent workspace	Initially make it a permission-controlled section of Platform Admin. Separate it if support operations grow.
Finance / settlement operations	Start as a restricted section of Platform Admin; separate it only if finance teams need independent workflows.
Public product storefront	Include it in the Customer Marketplace. Public browsing does not require a separate application.
Plugin developer portal	Add later if third-party developers need to register, test, publish and maintain plugins. This is different from a vendor installing plugins.

A key distinction: the vendor plugin portal is for using plugins; a plugin developer portal is for building and publishing plugins. They serve different actors.

5. How this applies to Ninepoint initially

You can start with all five experiences designed, but implement only the necessary functionality in each.

Build first

Customer marketplace, vendor operations, vendor plugin management, and the essential Platform Admin and Organization Admin screens.

Make operational

Product publishing, vendor-specific inventory, cart, checkout, payment, fulfillment, returns, billing, settlements and audit history.

Defer

Independent logistics, customer support, finance and plugin developer portals until there is a real need for separate workspaces.

One architectural point to settle before designing navigation: Ninepoint can be the organization operating the marketplace without making Platform Admin and Organization Admin the same role. A person may hold both roles, but the UI should make the active context clear and Core must enforce the distinct permissions.

My recommendation is to plan for five logical portal experiences, but only one frontend application initially. This gives you clear boundaries without the overhead of maintaining five

separate applications. Your next concrete step should be to define the sidebar navigation, pages and API dependencies for each portal, beginning with Platform Admin and Vendor.

Test cases:

Adan Pradan — Comprehensive Portal Test Case Specification

This test plan is for your multitenant e-commerce platform using Core (FastAPI), UI (Next.js), Temporal, Keycloak and PostgreSQL.

It covers the five portal experiences we defined:

1. Platform Admin
2. Organization / Tenant Admin
3. Vendor
4. Vendor Plugin Management
5. Customer Marketplace & Account

It also includes end-to-end commerce testing, cross-tenant authorization, payments, inventory, billing, refunds, settlement, Temporal workflows, database integrity, logs, security and deployment.

Your existing Identity and Authorization, organization/vendor upgrade workflows and vendor plugin portal should be regression-tested, while the new operational functionality is tested as it is implemented.

Important: This is a test specification, not a claim that these features or test cases have already been implemented. Some tests will be marked not applicable until the corresponding feature exists.

1. Test execution conventions

1.1 Test environment and sample data

Use a dedicated test environment with a payment gateway sandbox, test email/SMS providers, and test-only organizations and users. Never run destructive tests against production data.

Test fixtures

Organization A — Ninepoint Solution and Services LLP

One organization admin and two organization members, with different permissions as required by the test.

Vendor A — Stationery and gifts

Products with stock, variants, shipping, cancellation, returns and refunds.

Vendor B — Software solutions

A license-based product, a downloadable product or service offering, according to supported product types.

Organization B and Vendor C — Isolation fixtures

A second tenant and vendor used to verify that data and actions cannot cross unauthorized boundaries.

Customer A and Customer B

Separate customer identities, addresses, carts, wishlists, orders and payment histories.

Use clearly distinguishable test records, for example:

- ,
- , ,
- ,
- , ,

1.2 Priority and execution status

Priority	Meaning
P0 — Critical	Security breach, incorrect money movement, cross-tenant access, data corruption or inability to complete essential purchases
P1 — High	Core business journey fails, such as checkout, fulfillment, return or vendor operations
P2 — Medium	Important usability, reporting, administrative or recovery behavior fails

P3 — Low	Cosmetic, minor convenience or non-blocking behavior

For each test, record the actual execution status separately: Not Run, Pass, Fail, Blocked or Not Applicable.

1.3 Standard acceptance criteria

Unless a test specifies otherwise:

- The UI displays the correct outcome and an understandable error when an operation fails.
- Core enforces the business rule, regardless of what the UI permits.
- PostgreSQL stores a consistent result without duplicate or orphaned business records.
- Authorization checks include resource ownership and tenant scope, not only the user's role.
- Long-running Temporal workflows reach a valid terminal state or remain safely recoverable.
- Sensitive actions are auditable, and logs contain correlation identifiers without credentials or payment secrets.

2. Portal 1 — Platform Admin

The Platform Admin manages the marketplace as a whole. Tests in this section should cover the dashboard, organizations, vendors, catalog moderation, orders, financial oversight, plugin governance, auditing and system health.

2.1 Authentication, access and dashboard

Test ID	Scenario and steps	Expected result	Priority
PA-001	Log in with an authorized Platform Admin account.	Admin dashboard loads and permitted navigation is visible.	P0
PA-002	Open without authentication.	Redirect to login or return an appropriate authentication error.	P0

PA-003	Log in as a customer and manually open an admin URL.	Access is denied by Core, even if the page URL is known.	P0
PA-004	Log in as a vendor and invoke an admin API directly.	API rejects the unauthorized operation.	P0
PA-005	Use an expired or invalid access token against an admin API.	Request is rejected; protected data is not returned.	P0
PA-006	Refresh the page on an authorized admin route.	Session handling works correctly without exposing protected data.	P1
PA-007	Log out and use browser Back to revisit an admin page.	Protected data cannot be accessed through the stale session or API.	P0
PA-008	Load dashboard with organizations, vendors, orders and financial data present.	Metrics reflect the defined reporting rules and underlying records.	P1
PA-009	Load dashboard when no orders or vendors exist.	Empty states appear; no divide-by-zero or rendering errors occur.	P2
PA-010	Filter dashboard by date range or status.	Results use the selected filters, timezone and documented date semantics.	P2
PA-011	Compare totals across dashboard widgets and detailed reports.	Totals reconcile, with any documented exclusions made clear.	P1

PA-012	Simulate a backend timeout on a dashboard widget.	UI shows an error or retry state rather than misleading zero values.	P1

2.2 Organization administration

Test ID	Scenario and steps	Expected result	Priority
PA-013	Open the organization list.	Only authorized organization information is displayed.	P1
PA-014	Search by organization name or identifier.	Matching records appear without exposing unauthorized fields.	P2
PA-015	Open an organization detail page.	Correct profile, lifecycle status and permitted summary data appear.	P1
PA-016	Review a pending organization registration or upgrade request.	Request details, status and permitted actions are available.	P1
PA-017	Approve an eligible request.	Status changes according to business rules and the action is audited.	P1
PA-018	Reject a request with a reason.	Rejection is recorded and the reason is available to authorized users.	P1
PA-019	Attempt to approve an already approved request.	Duplicate approval is prevented or handled idempotently.	P1

PA-020	Attempt an invalid lifecycle transition.	Core rejects the action and leaves the prior state intact.	P1
PA-021	Suspend an organization with no outstanding activity.	Suspension follows the configured policy and is auditable.	P1
PA-022	Suspend an organization with open orders or unsettled balances.	Policy is enforced; unresolved activity is not silently discarded.	P0
PA-023	Reactivate an eligible suspended organization.	Access is restored according to policy without duplicating resources.	P1
PA-024	Attempt to modify an organization field outside the admin's permissions.	Core denies the change.	P0

2.3 Vendor administration

Test ID	Scenario and steps	Expected result	Priority
PA-025	List vendors across the marketplace.	Authorized cross-tenant vendor overview is displayed.	P1
PA-026	Open a vendor's profile and lifecycle history.	Correct vendor information and audit history are displayed.	P1
PA-027	Review and approve an eligible vendor application.	Vendor transitions to the configured approved/active state.	P1

PA-028	Reject a vendor application with a reason.	Request is rejected and the reason is recorded.	P1
PA-029	Suspend a vendor with active listings.	Listings and purchasing behavior follow the suspension policy.	P0
PA-030	Suspend a vendor with pending shipments and refunds.	Existing customer obligations remain visible and actionable.	P0
PA-031	Reactivate a suspended vendor.	Eligible capabilities are restored without duplicating products or permissions.	P1
PA-032	Change a vendor's commission policy.	New policy applies according to its effective date and scope; historical settlements remain explainable.	P0
PA-033	Change a vendor's payout details.	Sensitive changes require appropriate authorization and verification.	P0
PA-034	Access one vendor's private details as an admin without the required permission.	Restricted fields are hidden or the operation is denied.	P0

2.4 Product catalog and moderation

Test ID	Scenario and steps	Expected result	Priority
PA-035	View products from multiple vendors.	Each product retains its vendor and tenant ownership.	P1

PA-036	Filter products by vendor, category and publication status.	Filters return the correct matching records.	P2
PA-037	Approve a pending product.	Product becomes publishable according to its lifecycle rules.	P1
PA-038	Reject a product with a moderation reason.	Product is not purchasable and the reason is recorded.	P1
PA-039	Remove a published product from sale.	New purchases are prevented while existing orders remain consistent.	P0
PA-040	Edit a global category or category mapping.	Existing products are handled according to defined validation and migration rules.	P1
PA-041	Attempt to publish a product missing required fields.	Publication is rejected with actionable validation errors.	P1
PA-042	Attempt to approve a product belonging to another tenant using a tampered identifier without sufficient authorization.	Core rejects the action.	P0

2.5 Orders, returns and disputes

Test ID	Scenario and steps	Expected result	Priority
PA-043	Search marketplace orders by order ID, customer, vendor or status.	Results are accurate and appropriately access-controlled.	P1

PA-044	Open a parent order with multiple vendor suborders.	Parent/child relationships and fulfillment states are correct.	P1
PA-045	Inspect a payment-pending order.	Payment status is distinct from order status.	P0
PA-046	Cancel an eligible order through an authorized admin action.	Defined cancellation rules, inventory compensation and refund behavior execute correctly.	P0
PA-047	Attempt to cancel an already delivered order using a cancellation action.	Invalid transition is rejected; return policy is used instead.	P1
PA-048	Review a pending refund.	Original payment, refund amount, customer, order and status are linked.	P0
PA-049	Approve an eligible refund.	Approval is recorded; actual refund processing follows the configured workflow.	P0
PA-050	Attempt a refund greater than the refundable balance.	Core rejects the request and financial records remain consistent.	P0
PA-051	Review a disputed return or chargeback.	Evidence, decisions and permitted resolution actions are available.	P1
PA-052	Attempt to perform a refund twice using duplicate requests.	No duplicate financial refund occurs.	P0

2.6 Platform finance, plugins and operations

Test ID	Scenario and steps	Expected result	Priority
PA-053	View gross sales, discounts, tax, refunds and platform revenue reports.	Each metric follows a documented accounting definition.	P0
PA-054	Compare vendor payables with settlement statements.	Totals reconcile or show explainable differences.	P0
PA-055	Review failed vendor payouts.	Failure reason, provider reference and retry/recovery options are visible.	P0
PA-056	Retry an eligible failed payout.	The original payout is reconciled before any new payout is initiated.	P0
PA-057	View plugin catalog and installed-plugin inventory.	Plugin version, status, permissions and ownership are correct.	P1
PA-058	Disable a malicious or faulty plugin.	Execution is blocked according to policy and the action is audited.	P0
PA-059	Search audit records by actor, resource, date and action.	Results are accurate and access-controlled.	P1
PA-060	Open a failed Temporal workflow or business operation.	Authorized operators can inspect safe diagnostics and follow documented recovery procedures.	P1

3. Portal 2 — Organization / Tenant Admin

This portal manages one organization and its authorized members. For your initial rollout, use Ninepoint as Organization A, with two organization members. The tests must distinguish the organization administrator from ordinary organization members.

3.1 Organization profile and configuration

Test ID	Scenario and steps	Expected result	Priority
TA-001	Log in as an authorized organization admin and open the organization dashboard.	Correct organization context and permitted navigation appear.	P0
TA-002	Open organization settings.	Only settings belonging to the authorized organization are shown.	P0
TA-003	Update permitted organization details.	Valid changes persist and appear after refresh.	P1
TA-004	Submit invalid or missing mandatory organization fields.	Validation errors appear; invalid changes are not saved.	P1
TA-005	Attempt to change an immutable organization identifier.	Core rejects the modification.	P1
TA-006	Tamper with the organization ID in a request.	Core denies cross-organization access or modification.	P0
TA-007	Open settings while the organization is suspended.	Read and write access follow the configured suspension policy.	P0

TA-008	Update permitted organization preferences, such as timezone or branding.	Changes apply only within the authorized scope.	P2

3.2 Member and role management

Test ID	Scenario and steps	Expected result	Priority
TA-009	View organization members.	Only authorized members of the current organization are shown.	P0
TA-010	Invite a new member using a valid email address.	Invitation is created and delivered through the configured mechanism.	P1
TA-011	Invite an existing member who already belongs to the organization.	Duplicate membership is prevented or handled idempotently.	P1
TA-012	Invite a user who already exists in Keycloak but is not a member of the organization.	Membership is established through the approved flow without duplicating identity records.	P1
TA-013	Invite a user with an invalid email address.	Validation rejects the request.	P2
TA-014	Invite a member with a role outside the inviter's delegation scope.	Core denies the assignment.	P0
TA-015	Assign an allowed organization-level role.	Permissions update correctly and are enforced by Core.	P0

TA-016	Change a member's role.	New permissions take effect according to the defined token/session and authorization-cache policy.	P0
TA-017	Deactivate a member.	Membership access is revoked; historical business records remain intact.	P0
TA-018	Remove or deactivate the last organization admin.	The system prevents accidental loss of administrative control or requires an approved transfer.	P0
TA-019	Attempt to manage a member from another organization.	Core denies the action.	P0
TA-020	Open a direct URL to a member-management page as a read-only member.	Page and APIs enforce the member's permissions.	P0
TA-021	Revoke a member's permissions while they have an active session.	Subsequent protected requests fail according to the documented revocation behavior.	P0
TA-022	Reinvite an expired or revoked invitation.	The new invitation follows the defined invitation lifecycle without creating duplicate active memberships.	P1

3.3 Vendor and store relationships

These tests depend on your actual tenancy model: a vendor may be an independent tenant, or an organization may have a relationship with multiple vendors. The expected access must match your existing authorization graph.

Test ID	Scenario and steps	Expected result	Priority
TA-023	View vendors associated with the organization.	Only permitted relationships are shown.	P1
TA-024	Initiate an authorized vendor onboarding or relationship request.	Request enters the correct workflow and records its initiator.	P1
TA-025	Open an existing vendor relationship.	Status, scope and permitted operations are displayed.	P1
TA-026	Attempt to modify a vendor's private profile without permission.	Core denies the operation.	P0
TA-027	Attempt to access a vendor that is not associated with the organization where association is required.	Access is denied.	P0
TA-028	Deactivate a vendor relationship while orders are outstanding.	The platform applies the defined policy without deleting order or financial history.	P0
TA-029	Change organization settings affecting vendor access.	Only eligible resources and permissions change; unrelated tenants remain unaffected.	P0

3.4 Organization reports and audit

Test ID	Scenario and steps	Expected result	Priority

TA-030	Open organization sales or operational reports.	Data is restricted to the authorized organization scope.	P0
TA-031	Filter a report by date, vendor or status.	The report respects filters and reporting timezone.	P1
TA-032	Export a report.	Export contains only permitted fields and authorized records.	P0
TA-033	Attempt to manipulate export filters to access another tenant's data.	Request is rejected or filtered to the authorized scope.	P0
TA-034	Review organization activity history.	Relevant membership, settings and administrative changes are traceable.	P1
TA-035	Attempt to delete an organization with orders or outstanding balances.	Deletion is blocked or follows an explicit offboarding and retention workflow.	P0

4. Portal 3 — Vendor

The Vendor Portal is central to your initial marketplace. Both the stationery/gifts vendor and software-solutions vendor need the appropriate functions, but their fulfillment requirements differ.

4.1 Vendor access and dashboard

Test ID	Scenario and steps	Expected result	Priority
VE-001	Log in as an active vendor user.	The correct vendor workspace opens.	P0

VE-002	Open the vendor dashboard with existing products and orders.	Metrics match the vendor's permitted records.	P1
VE-003	Log in as a suspended vendor.	Restricted operations are blocked according to suspension policy.	P0
VE-004	Open another vendor's dashboard by changing the URL identifier.	Core denies access.	P0
VE-005	Open a vendor route as a customer.	Protected vendor operations are inaccessible.	P0
VE-006	Load the dashboard with no products or orders.	Empty states and onboarding guidance appear.	P2
VE-007	Filter dashboard metrics by date or status.	Results match the defined metric semantics.	P2
VE-008	Refresh or reopen a dashboard after a backend error.	Data is reloaded safely and the UI recovers.	P2

4.2 Product catalog management

Test ID	Scenario and steps	Expected result	Priority
VE-009	Create a product with valid required fields.	Product is stored under the correct vendor and tenant.	P0
VE-010	Create a product with missing name, price or required type fields.	Validation fails; no invalid published product is created.	P1

VE-011	Create a product with negative price or invalid currency.	Core rejects invalid values.	P0
VE-012	Create a product with duplicate SKU within the required uniqueness scope.	Duplicate is rejected or handled under the documented SKU policy.	P1
VE-013	Create a product with variants such as size or color.	Each variant has correct SKU, price and inventory identity.	P1
VE-014	Upload valid product images.	Images are stored and rendered correctly.	P2
VE-015	Upload unsupported, oversized or malicious file content.	Upload is rejected or safely processed; content is not executed.	P0
VE-016	Save a product as draft.	Draft is visible to authorized vendor users but not purchasable publicly.	P1
VE-017	Submit a product for approval where moderation is required.	Correct approval workflow is triggered.	P1
VE-018	Publish an approved product.	Product becomes discoverable according to publication rules.	P0
VE-019	Edit a published product's description.	Updated content is validated and displayed correctly.	P1
VE-020	Change a product price while a customer has it in the cart.	Checkout uses current authoritative pricing and informs the customer of material changes.	P0

VE-021	Deactivate a product with no active orders.	Product is removed from new purchase eligibility.	P1
VE-022	Deactivate a product with existing orders.	Existing obligations and history remain intact.	P0
VE-023	Attempt to edit another vendor's product by supplying its ID.	Core denies the update.	P0
VE-024	Bulk import a valid product file.	Valid records are imported and a result summary is available.	P2
VE-025	Bulk import a file containing both valid and invalid rows.	Partial-success or all-or-nothing behavior follows the documented import contract; row-level errors are clear.	P1
VE-026	Submit the same import job twice.	Duplicate products are not unintentionally created.	P1

4.3 Vendor inventory

Test ID	Scenario and steps	Expected result	Priority
VE-027	View stock for a vendor-owned SKU.	Correct on-hand, reserved and available quantities are shown.	P0
VE-028	Receive or add stock with a valid quantity.	Stock increases and a stock movement is recorded.	P1

VE-029	Attempt to add a negative or invalid quantity through a receipt operation.	Core rejects the operation.	P1
VE-030	Adjust stock with a valid reason and authorized permission.	Stock changes and the adjustment is auditable.	P1
VE-031	Attempt a stock adjustment without the required permission.	Core denies the change.	P0
VE-032	Sell the last available unit.	Reservation succeeds for only one competing checkout.	P0
VE-033	Simulate two customers purchasing the last unit concurrently.	Stock is not oversold.	P0
VE-034	Cancel an order before fulfillment.	Reservation is released exactly once according to policy.	P0
VE-035	Complete a shipment.	Inventory is decremented according to the chosen reservation/stock accounting model, without double deduction.	P0
VE-036	Receive returned goods and mark them sellable.	Only eligible inspected quantities return to available stock.	P0
VE-037	Receive returned goods marked damaged.	Damaged goods do not become available for sale.	P1

VE-038	Configure a low-stock threshold.	Alerts are triggered at the defined threshold without uncontrolled duplication.	P2
VE-039	Attempt to view or adjust another vendor's inventory.	Core denies access.	P0
VE-040	Attempt an invalid stock movement or replay the same inventory event.	Database constraints and idempotency prevent inconsistent or duplicate movements.	P0

4.4 Vendor order management

Test ID	Scenario and steps	Expected result	Priority
VE-041	Open the incoming-order list.	Only authorized vendor suborders appear.	P0
VE-042	Open an order detail.	Items, quantities, shipping information and permitted customer details are accurate.	P1
VE-043	Accept an eligible paid order.	Order transitions to the permitted state and relevant events are recorded.	P0
VE-044	Reject an order with a required reason.	Defined cancellation/refund policy executes; vendor cannot silently discard the order.	P0
VE-045	Attempt to accept an unpaid order when policy prohibits it.	Operation is rejected.	P0

VE-046	Accept an already accepted order twice.	No duplicate fulfillment operation is created.	P1
VE-047	Pack an eligible order.	Packing state and associated information are persisted.	P1
VE-048	Dispatch an eligible order with valid tracking information.	Shipment is linked to the correct vendor suborder.	P0
VE-049	Dispatch an order without required shipping details.	Validation fails.	P1
VE-050	Attempt to dispatch another vendor's order.	Core denies access.	P0
VE-051	Fulfill only some items in an order.	Partial fulfillment is represented accurately; remaining quantities stay actionable.	P1
VE-052	Receive a duplicate shipment status event.	Order/shipment state and notifications are not duplicated incorrectly.	P0
VE-053	Mark an order delivered without satisfying required transition rules.	Core rejects the invalid transition.	P1
VE-054	Vendor cancels after dispatch.	Cancellation is blocked or follows the explicit in-transit return/refund process.	P0

4.5 Vendor returns, refunds and customer disputes

Test ID	Scenario and steps	Expected result	Priority
VE-055	View return requests for the vendor's suborders.	Only authorized requests are visible.	P1
VE-056	Approve an eligible return.	Return instructions or pickup workflow are initiated.	P1
VE-057	Reject a return with a reason.	Reason is recorded and customer receives the configured notification.	P1
VE-058	Attempt to process a return outside the allowed window.	Eligibility rules are enforced.	P1
VE-059	Inspect a returned item and record its condition.	Inspection outcome is stored and linked to the return item.	P1
VE-060	Approve a replacement.	Replacement order or fulfillment record is created without unintentionally double-charging the customer.	P1
VE-061	Request a refund for an amount exceeding the eligible balance.	Core rejects the request.	P0
VE-062	Attempt to issue a refund directly without the required authority or payment workflow.	Core denies the action.	P0
VE-063	Open a customer dispute.	Vendor sees only the information and evidence permitted by policy.	P1

VE-064	Submit duplicate return approval or inspection requests.	Repeated requests do not duplicate refunds, pickups or inventory changes.	P0

4.6 Vendor finance, team and reporting

Test ID	Scenario and steps	Expected result	Priority
VE-065	View vendor sales and order reports.	Metrics are scoped to the vendor and follow defined accounting rules.	P1
VE-066	View commission deducted from an order.	Commission basis, rate and amount are explainable.	P0
VE-067	View pending and completed settlements.	Vendor payable, holdbacks, adjustments and payouts are correctly represented.	P0
VE-068	Download a settlement statement.	Statement matches the underlying ledger and settlement period.	P0
VE-069	View a failed payout.	Status, reason and permitted next steps are visible.	P1
VE-070	Update vendor profile or payout details.	Authorized changes are validated, protected and audited.	P0
VE-071	Invite vendor staff with a limited role.	The new user can perform only the permitted actions.	P1

VE-072	Revoke a vendor staff member's access.	Subsequent protected requests are denied according to revocation policy.	P0
VE-073	Export a vendor report and attempt to access another vendor's data through modified parameters.	Export remains restricted to authorized vendor scope.	P0

5. Portal 4 — Vendor Plugin Management

This portal is already implemented in some form, so use these cases for regression and expansion. Separate vendor plugin management from platform-wide plugin governance and, if introduced later, third-party developer publishing.

5.1 Plugin discovery and eligibility

Test ID	Scenario and steps	Expected result	Priority
VP-001	Open the plugin catalog as an authorized vendor user.	Only plugins available for discovery are shown.	P1
VP-002	Search and filter plugins by category or capability.	Results match filters and pagination.	P2
VP-003	Open plugin details.	Description, version, compatibility, permissions and pricing/terms where applicable are visible.	P1
VP-004	Attempt to install a plugin incompatible with the vendor's plan or product type.	Installation is blocked with a clear reason.	P1

VP-005	Attempt to install a suspended or withdrawn plugin.	Installation is blocked.	P0
VP-006	Attempt to access a plugin unavailable to the current vendor.	Core denies the action.	P0

5.2 Installation, configuration and lifecycle

Test ID	Scenario and steps	Expected result	Priority
VP-007	Install an eligible plugin.	Installation record and status are persisted under the correct vendor/tenant scope.	P0
VP-008	Submit two installation requests simultaneously.	Only one effective installation is created unless multiple instances are explicitly supported.	P0
VP-009	Retry an installation after a timeout.	Existing installation state is reconciled; duplicate side effects are prevented.	P0
VP-010	Configure valid plugin settings.	Settings are validated and saved securely.	P1
VP-011	Submit invalid or missing required settings.	Configuration fails with actionable validation errors.	P1
VP-012	Update a secret or API credential.	Secret is stored using the approved secret-handling approach and never returned in plaintext unnecessarily.	P0

VP-013	Read another vendor's plugin configuration using a tampered identifier.	Core denies access.	P0
VP-014	Enable a successfully configured plugin.	Plugin enters the enabled state only after required validation.	P0
VP-015	Enable a plugin with missing permissions or invalid dependencies.	Enablement is blocked.	P1
VP-016	Disable an enabled plugin.	New execution is prevented according to the defined lifecycle; in-flight work is handled safely.	P0
VP-017	Uninstall a plugin that has active workflows or stored data.	Required cleanup, retention and dependency rules are applied; data is not silently corrupted.	P0
VP-018	Reinstall a previously removed plugin.	Reinstallation follows the defined data-retention and configuration policy.	P1
VP-019	Upgrade a plugin to a compatible version.	Version changes are tracked and migration/rollback requirements are enforced.	P1
VP-020	Attempt an unsupported downgrade.	Core blocks the downgrade or uses an explicitly supported migration procedure.	P1

5.3 Plugin execution, permissions and monitoring

Test ID	Scenario and steps	Expected result	Priority

VP-021	Run a plugin with valid inputs and permissions.	Expected output or side effect occurs within the defined execution contract.	P1
VP-022	Run a plugin without a required permission.	Execution is denied.	P0
VP-023	Attempt to use a plugin to access another vendor's resources.	Core's resource authorization prevents access.	P0
VP-024	Submit malformed plugin input.	Input validation fails safely.	P1
VP-025	Simulate plugin execution timeout.	Timeout is recorded and the system remains responsive.	P1
VP-026	Simulate a transient plugin failure.	Retry policy is applied only where safe; duplicate side effects are prevented.	P1
VP-027	Simulate a permanent plugin failure.	Plugin status/error details are available and repeated uncontrolled retries stop.	P1
VP-028	Inspect plugin logs.	Authorized users see appropriate diagnostics without secrets or other vendors' data.	P0
VP-029	Inspect plugin health after disabling it.	Status reflects the actual disabled state.	P2

VP-030	Attempt to bypass plugin permissions by calling its execution endpoint directly.	Core rejects unauthorized execution.	P0
VP-031	Attempt plugin execution after vendor suspension.	Execution follows suspension policy and cannot bypass vendor restrictions.	P0
VP-032	Verify plugin activity is associated with the initiating actor and workflow.	Audit records provide traceability.	P1

6. Portal 5 — Customer Marketplace & Account

This portal is the main shopping experience. Test public browsing, customer accounts, shortlist, wishlist, cart, pricing, checkout, payments, orders, shipping, cancellation, returns, refunds, invoices, reviews and support.

6.1 Public storefront and product discovery

Test ID	Scenario and steps	Expected result	Priority
CU-001	Open the public marketplace without logging in.	Publicly available products and categories load if guest browsing is supported.	P1
CU-002	Open the marketplace with no published products.	A clear empty state appears; the page does not fail.	P2

CU-003	Browse products from both initial vendors.	Only eligible, published and available-for-discovery products appear.	P0
CU-004	Search by product name.	Relevant products are returned.	P1
CU-005	Search by SKU or supported product identifier.	Results follow the documented search behavior.	P2
CU-006	Search with no matches.	An informative empty state appears.	P2
CU-007	Search with punctuation, Unicode and unusual whitespace.	Input is handled safely; no errors or query injection occur.	P1
CU-008	Filter by category, price range, vendor or product type.	Results satisfy all selected filters.	P1
CU-009	Sort by price or another supported criterion.	Ordering is correct and stable for equivalent values.	P2
CU-010	Open a product detail page.	Description, images, variants, price, vendor and fulfillment details are correct.	P0
CU-011	Open a draft, rejected or deactivated product using its direct URL.	Unpublished product details are not exposed to unauthorized visitors or made purchasable.	P0
CU-012	Open a product that has become out of stock.	Current availability is shown and purchase is prevented as appropriate.	P0

CU-013	Open a product with several variants.	Each selected variant displays its correct price, SKU and availability.	P1
CU-014	Navigate through product pages using browser Back and Forward.	Navigation state behaves consistently.	P2
CU-015	Load a product with a missing image or unavailable media resource.	A fallback image or safe error state appears.	P2
CU-016	Attempt to manipulate product IDs to access restricted vendor-only data.	Private vendor data remains protected.	P0

6.2 Customer registration, login and profile

Test ID	Scenario and steps	Expected result	Priority
CU-017	Register a new customer through the supported identity flow.	A valid identity and customer profile are established without duplicate records.	P1
CU-018	Register using an existing identity.	Duplicate registration is prevented or the existing account is handled correctly.	P1
CU-019	Log in through Keycloak.	Successful authentication returns the customer to an appropriate destination.	P0

CU-020	Attempt login with invalid credentials.	Authentication fails without exposing sensitive details.	P1
CU-021	Open the customer account without authentication.	Login is required for protected account operations.	P0
CU-022	Log out and revisit the account.	Protected data is inaccessible after logout.	P0
CU-023	Update permitted profile fields.	Changes persist and are validated.	P1
CU-024	Submit malformed profile data.	Invalid data is rejected.	P2
CU-025	Attempt to change protected identity fields without the required verification.	Core denies the change or requires the defined verification flow.	P0
CU-026	Deactivate the customer account.	Account lifecycle and retention rules are followed without corrupting orders or financial records.	P1
CU-027	Access another customer's profile through a modified API identifier.	Core denies access.	P0

6.3 Shortlist and wishlist

Test ID	Scenario and steps	Expected result	Priority

CU-028	Add a product to a shortlist.	Product is saved to the intended shortlist.	P2
CU-029	Remove a product from the shortlist.	It is removed without affecting the cart or wishlist.	P2
CU-030	Add the same product to a shortlist twice.	Duplicates are prevented or handled according to the defined policy.	P2
CU-031	Add a product to the wishlist.	Wishlist entry is persisted for the correct customer.	P1
CU-032	Remove a product from the wishlist.	The entry is removed and the updated state is shown.	P1
CU-033	Add an out-of-stock product to the wishlist.	The system allows or blocks saving according to policy; it never reserves stock.	P2
CU-034	View the wishlist after logging out and back in.	Saved items persist for the same customer.	P1
CU-035	Attempt to read another customer's wishlist.	Core denies access.	P0
CU-036	Move a wishlist item to the cart.	The item is added with a valid selected variant and quantity; wishlist behavior follows the documented policy.	P1
CU-037	Open a wishlist item whose product is discontinued.	The customer sees an unavailable-product state and cannot purchase it.	P1

CU-038	Add and remove the same item concurrently from different sessions.	Final state remains consistent and does not create corrupted records.	P2

6.4 Cart management and pricing

Test ID	Scenario and steps	Expected result	Priority
CU-039	Add an eligible product to the cart.	Correct product, variant, unit price snapshot and quantity are represented.	P0
CU-040	Add a second product from the same vendor.	Both items appear correctly.	P1
CU-041	Add products from both vendors.	The cart supports multiple vendor suborders or allocations as designed.	P0
CU-042	Increase item quantity within available stock and configured limits.	Quantity and displayed subtotal update correctly.	P0
CU-043	Decrease item quantity to a valid positive quantity.	Quantity and totals recalculate correctly.	P1
CU-044	Attempt quantity zero or a negative quantity.	The system rejects invalid values or uses the explicit remove-item action.	P1
CU-045	Set quantity above available stock.	The operation is rejected or quantity is capped with clear feedback.	P0

CU-046	Remove an item from the cart.	Item is removed and totals update.	P1
CU-047	Remove the last item.	Empty-cart state appears and checkout is disabled.	P1
CU-048	Refresh the browser after modifying the cart.	Cart persistence matches guest/account policy.	P1
CU-049	Log in with a cart created as a guest.	Guest cart merge follows the defined policy without unintended duplication.	P2
CU-050	Apply a valid coupon.	Eligible discount is calculated correctly.	P1
CU-051	Apply an expired, invalid or ineligible coupon.	Coupon is rejected and totals remain correct.	P1
CU-052	Apply the same single-use coupon twice or exceed its usage limit.	Coupon limits are enforced server-side.	P0
CU-053	Recalculate totals after a price change.	Current authoritative pricing is used; the customer is informed of relevant changes.	P0
CU-054	Recalculate totals after stock changes.	Ineligible quantities cannot proceed to checkout.	P0
CU-055	Verify subtotal, discount, tax, shipping and fees.	Displayed total matches Core's authoritative pricing breakdown.	P0

CU-056	Attempt to alter unit price or payable amount through browser tools/API payloads.	Server ignores or rejects client-supplied financial amounts.	P0
CU-057	Attempt to create overflow quantities or excessively large carts.	Limits and numeric validation prevent corruption or resource exhaustion.	P1

6.5 Address, checkout and payment

Test ID	Scenario and steps	Expected result	Priority
CU-058	Add a valid delivery address.	Address is saved to the correct customer account.	P1
CU-059	Submit an incomplete or invalid address.	Validation rejects the address.	P1
CU-060	Edit and delete an address that is not in active use.	Changes persist and the address list updates.	P2
CU-061	Attempt to edit another customer's address.	Core denies access.	P0
CU-062	Select a valid delivery address at checkout.	Address is correctly associated with the order snapshot.	P0
CU-063	Select an address that is ineligible for a product's shipping region.	Checkout explains the restriction and prevents invalid fulfillment.	P1

CU-064	Check out with an empty cart.	Checkout is blocked.	P1
CU-065	Check out with a product that is no longer available.	Checkout fails safely; no payable order is confirmed.	P0
CU-066	Check out with a price that has changed.	Updated price is displayed and reconfirmation is requested when required.	P0
CU-067	Submit checkout twice rapidly.	Duplicate order/payment side effects are prevented using idempotency and transaction rules.	P0
CU-068	Start payment with a valid sandbox payment method.	A payment attempt and its associated order state are recorded.	P0
CU-069	Simulate gateway payment success.	Trusted confirmation is verified and order progression follows policy.	P0
CU-070	Simulate gateway payment failure.	Order/payment state remains accurate and retry or cancellation follows policy.	P0
CU-071	Simulate a payment timeout with an unknown gateway outcome.	System reconciles payment status before charging again or releasing inventory unsafely.	P0
CU-072	Send a forged or unsigned payment webhook.	Webhook is rejected and order state is not changed.	P0
CU-073	Deliver the same valid webhook more than once.	Payment, inventory and order transitions are applied idempotently.	P0

CU-074	Receive a delayed payment success after a timeout or cancellation decision.	Defined recovery/reconciliation rules prevent inconsistent fulfillment or duplicate refunds.	P0
CU-075	Cancel a payment attempt while the gateway is processing it.	Ambiguous outcomes are reconciled; cancellation does not assume the charge failed.	P0
CU-076	Retry a failed payment.	A safe new attempt is created or the existing attempt is reconciled without duplicate charges.	P0
CU-077	Pay for items from two vendors.	Parent order, vendor suborders and financial allocations reconcile to the payment.	P0
CU-078	One vendor cannot fulfill a suborder after the overall payment succeeds.	The configured partial-fulfillment, cancellation and refund policy is applied.	P0
CU-079	Open the payment success URL directly without a real payment.	Order is not marked paid based solely on the URL.	P0

6.6 Customer orders, shipment and digital delivery

Test ID	Scenario and steps	Expected result	Priority
CU-080	Open order history.	Only the authenticated customer's orders are shown.	P0

CU-081	Open an order detail.	Items, totals, payment status, vendor suborders and permitted tracking information are accurate.	P0
CU-082	Open another customer's order using a modified identifier.	Core denies access.	P0
CU-083	Track an order before dispatch.	UI shows the correct pending-fulfillment state.	P1
CU-084	Track a dispatched physical order.	Carrier and tracking details are displayed when available.	P1
CU-085	Process partial shipment of a multi-item order.	Shipped and remaining items are clearly distinguished.	P1
CU-086	Process delivery confirmation.	Customer timeline reflects the verified shipment state.	P1
CU-087	Simulate delayed, lost or failed delivery.	Status and support/escalation options reflect the event.	P1
CU-088	Open a digital product order after payment confirmation.	Authorized digital delivery or license activation is available according to the product contract.	P0
CU-089	Attempt to access another customer's digital download or license.	Access is denied.	P0

CU-090	Reopen a digital download after link expiry or entitlement revocation.	Access follows expiry, revocation and refund policies.	P1

6.7 Cancellation, returns, refunds and replacements

Test ID	Scenario and steps	Expected result	Priority
CU-091	Cancel an eligible order before fulfillment.	Cancellation is accepted, inventory is released safely and refund handling is initiated where applicable.	P0
CU-092	Cancel an order after dispatch.	Direct cancellation is blocked or the configured return/recall process is offered.	P0
CU-093	Cancel one vendor suborder in a multi-vendor order.	Other suborders remain consistent; financial allocations and customer totals are correct.	P0
CU-094	Attempt to cancel an already cancelled order.	Duplicate cancellation side effects do not occur.	P1
CU-095	Request a return within the permitted window.	Request is accepted and linked to the correct order items.	P0
CU-096	Request a return outside the permitted window.	Request is rejected or routed through an explicitly supported exception process.	P1
CU-097	Return only one item from a multi-item order.	Return and refund eligibility are calculated for the selected items only.	P0

CU-098	Upload required return evidence.	File validation, access restrictions and retention rules are enforced.	P1
CU-099	Track return approval, pickup and inspection.	Customer sees the correct return lifecycle and next step.	P1
CU-100	Receive return rejection.	Reason and any appeal/support options are displayed according to policy.	P1
CU-101	Request a replacement.	Replacement follows its own fulfillment lifecycle and is not accidentally billed as a new purchase.	P1
CU-102	View a pending refund.	Customer sees the correct amount, original payment reference and status.	P0
CU-103	Process a successful partial refund.	Refunded amount and remaining refundable balance are accurate.	P0
CU-104	Simulate a failed refund.	Failure is visible and recoverable; the system does not falsely report success.	P0
CU-105	Submit a duplicate refund request.	No duplicate refund is issued.	P0
CU-106	Verify refund amount after discounts, shipping and taxes.	Refund follows the documented allocation policy and reconciles with the order.	P0
CU-107	Check a refund after a vendor has been settled.	Vendor ledger adjustments and recovery rules are applied correctly.	P0

CU-108	Request a refund for a digital product.	Eligibility and entitlement handling follow the digital product's refund policy.	P1

6.8 Billing, notifications, reviews and support

Test ID	Scenario and steps	Expected result	Priority
CU-109	View the order invoice or receipt.	Correct order reference, seller/issuer information, amounts, currency and required tax details are displayed.	P0
CU-110	Download an invoice.	Download succeeds and contains the same finalized data as the issued document.	P1
CU-111	Attempt to download another customer's invoice.	Core denies access.	P0
CU-112	View payment and refund history.	Entries match payment records and financial outcomes.	P0
CU-113	Receive order confirmation notification.	Message contains correct order information and no sensitive payment credentials.	P1
CU-114	Simulate notification delivery failure.	Failure is recorded and retry policy is applied.	P2
CU-115	Review an eligible purchased product.	Review is linked to the correct purchase according to verification policy.	P2

CU-116	Submit duplicate or abusive reviews.	Duplicate, moderation and abuse policies are enforced.	P2
CU-117	Create a support ticket against an order.	Ticket is associated with the authenticated customer and relevant order.	P1
CU-118	Attach a file to a support ticket.	File restrictions and access controls are enforced.	P1
CU-119	Read another customer's ticket.	Access is denied.	P0
CU-120	Close and reopen a support ticket where permitted.	Lifecycle transitions and audit history are correct.	P2

7. Cross-portal end-to-end commerce tests

Individual portal tests are not sufficient. These cases verify that the customer, vendor, admin, payment provider, database and Temporal workflows all agree.

Test ID	End-to-end scenario	Expected result	Priority
E2E-001	Vendor creates, publishes and stocks a notebook; customer buys it; vendor dispatches it; shipment is delivered.	Catalog, inventory, order, payment, shipment and customer views agree.	P0

E2E-002	Customer purchases products from both vendors in one cart.	Parent order and vendor suborders are created with correct item ownership and allocations.	P0
E2E-003	Customer buys the last unit while another customer checks out concurrently.	Only valid reservations succeed.	P0
E2E-004	Payment succeeds but the browser never receives the success redirect.	Server-side confirmation eventually establishes the correct payment/order state.	P0
E2E-005	Payment webhook arrives before a delayed response from the gateway request.	State transitions remain correct and idempotent.	P0
E2E-006	Payment times out and later reconciles as successful.	No duplicate charge; fulfillment or compensation follows policy.	P0
E2E-007	Vendor rejects one suborder after customer payment.	Rejection, inventory release and partial/full refund follow policy.	P0
E2E-008	One of two vendor suborders is delivered and the other is delayed.	Each suborder tracks independently while the parent order remains accurate.	P0
E2E-009	Customer returns one item from a delivered order.	Return, inventory disposition, refund and financial ledger reconcile.	P0
E2E-010	Customer receives a refund after vendor settlement.	Appropriate vendor balance adjustment or recovery is recorded.	P0

E2E-011	Software purchase triggers license provisioning.	Entitlement is issued only after verified payment and is linked to the customer/order.	P0
E2E-012	Temporal worker restarts during payment recovery.	Workflow resumes or reconciles safely without duplicate money movement.	P0
E2E-013	Temporal worker restarts during return/refund processing.	Workflow resumes safely with accurate return, refund and inventory states.	P0
E2E-014	Platform admin suspends a vendor with open orders.	New purchases follow suspension policy; existing customer obligations remain managed.	P0
E2E-015	Organization member loses access during an active session.	Subsequent protected operations enforce the revocation policy.	P0
E2E-016	A vendor attempts to manipulate another vendor's product/order ID.	No cross-vendor data access or mutation occurs.	P0
E2E-017	A customer retries checkout while the first request is still running.	No duplicate payable order or charge is created.	P0
E2E-018	Notification service is unavailable after successful delivery.	Commerce state remains correct; notification retries independently.	P1
E2E-019	Database transaction fails while writing order records.	Partial order creation is rolled back or reconciled according to the transaction/outbox design.	P0

E2E-020	Admin, vendor and customer inspect the same order from their own portals.	Each sees a consistent lifecycle with only their permitted information and actions.	P0

8. Identity, authorization and multitenancy — cross-portal tests

These tests apply to all five portals and are among the most important for your architecture. A role check alone is not enough; access must also be checked against the specific resource and its tenant/vendor/customer ownership.

Test ID	Scenario and steps	Expected result	Priority
SEC-001	Access a protected API without a token.	Request is rejected.	P0
SEC-002	Use an expired, malformed or invalidly signed token.	Request is rejected.	P0
SEC-003	Use a token issued for an untrusted issuer or wrong audience.	Request is rejected where issuer/audience validation is required.	P0
SEC-004	Log in as a customer and invoke a vendor API directly.	Request is denied.	P0
SEC-005	Log in as a vendor and invoke a platform-admin API.	Request is denied.	P0
SEC-006	Use an authorized role but an unauthorized resource ID.	Resource-level authorization denies access.	P0

SEC-007	Modify , , or in a request.	Core derives or validates scope independently; unauthorized access is denied.	P0
SEC-008	Attempt cross-tenant list, search, count and aggregation queries.	Results, totals and metadata do not leak unauthorized records.	P0
SEC-009	Attempt cross-tenant access through export, download or report APIs.	Unauthorized records cannot be exported.	P0
SEC-010	Attempt cross-tenant access through a background job or Temporal workflow.	Workflow activity validates the intended scope and resource authorization.	P0
SEC-011	Change permissions while a user has an active session.	Authorization follows the documented revocation and caching policy.	P0
SEC-012	Remove an authorization relationship in SpiceDB.	Subsequent requests follow the configured consistency guarantees and deny revoked access as required.	P0
SEC-013	SpiceDB is unavailable.	Protected operations fail safely rather than bypassing authorization.	P0
SEC-014	Keycloak is unavailable during a new login.	Login fails safely; existing sessions follow the defined token-validation policy.	P0

SEC-015	Attempt to assign a platform-admin role from an organization-admin interface.	Role escalation is denied.	P0
SEC-016	Attempt privilege escalation through request fields, mass assignment or manipulated role claims.	Protected fields and permissions cannot be modified by the caller.	P0
SEC-017	Access a file or object using a guessed storage URL.	Private content remains inaccessible without valid authorization.	P0
SEC-018	Attempt SQL injection in search, filtering or identifiers.	Queries remain parameterized; no unauthorized data is returned.	P0
SEC-019	Submit XSS payloads in product descriptions, reviews, plugin metadata or support tickets.	Content is safely encoded or sanitized.	P0
SEC-020	Submit CSRF attempts against cookie-authenticated state-changing operations, if applicable.	Required CSRF protections prevent unauthorized changes.	P0
SEC-021	Send requests beyond configured rate limits.	Rate limiting works without bypassing authorization or corrupting data.	P1
SEC-022	Attempt login or password-reset abuse.	Authentication protection and account recovery policies are enforced.	P1

SEC-023	Attempt to replay a sensitive administrative request.	Idempotency or business rules prevent repeated harmful side effects.	P0
SEC-024	Inspect error messages for database, token, secret or internal stack details.	External responses disclose only safe diagnostics.	P1
SEC-025	Attempt to access secrets in plugin settings, application configuration or logs.	Secrets are appropriately protected and access is restricted.	P0

Additional isolation invariants

Automate these as database/API tests:

- Every tenant-owned resource is associated with the correct tenant or organization.
- Vendor-owned products, inventory, orders and financial records cannot be mutated by another vendor.
- Customer-owned carts, wishlists, addresses, orders, invoices and support tickets cannot be accessed by another customer.
- Every vendor suborder item belongs to the correct parent order and vendor.
- A payment, refund and settlement cannot be associated with an unrelated order or vendor.
- Administrative cross-tenant access is granted only to explicitly authorized roles and is auditable.

9. PostgreSQL, migrations and data integrity tests

Test ID	Scenario and steps	Expected result	Priority
DB-001	Create each required entity through Core.	Valid rows and relationships are persisted correctly.	P0

DB-002	Submit a record referencing a nonexistent tenant, vendor or product.	Foreign keys or business validation reject invalid references.	P0
DB-003	Attempt duplicate order idempotency keys or duplicate payment references.	Uniqueness constraints prevent unintended duplicates.	P0
DB-004	Attempt negative quantities, invalid currency values or impossible financial amounts.	Database constraints and Core validation reject invalid data.	P0
DB-005	Cause an error during a multi-table order transaction.	Transaction rolls back incomplete changes.	P0
DB-006	Create concurrent reservations for the last available unit.	Transaction and locking rules prevent overselling.	P0
DB-007	Apply a new Alembic migration to a representative previous schema version.	Migration succeeds and data remains correct.	P0
DB-008	Attempt migration with invalid existing data.	Migration fails safely with a diagnosable error rather than silently corrupting records.	P0
DB-009	Verify downgrade or rollback procedures where supported.	Recovery behavior matches the documented migration strategy.	P1
DB-010	Rebuild the environment from migrations on an empty database.	Schema and required seed data can be recreated deterministically.	P0

DB-011	Restore a backup into an isolated environment.	Data can be restored and integrity checks pass.	P0
DB-012	Delete a product with historical order items.	Referential integrity and historical records are preserved according to retention rules.	P0
DB-013	Delete or deactivate a customer/vendor with outstanding financial records.	Lifecycle rules prevent loss of required business records.	P0
DB-014	Inspect tenant filtering in ORM queries and raw SQL paths.	All access paths apply the required tenant scope.	P0
DB-015	Run a migration during a controlled deployment.	Application and schema compatibility follows the rollout plan.	P1
DB-016	Inspect indexes for product search, order lists and tenant-scoped queries.	Representative queries perform within agreed thresholds.	P1
DB-017	Attempt a concurrent update to the same payment/refund record.	State and ledger invariants remain valid.	P0
DB-018	Replay an outbox event after a worker restart.	Consumers deduplicate events and do not repeat financial side effects.	P0

10. Temporal workflow and asynchronous processing tests

These tests apply wherever Temporal is used for organization upgrades, payment recovery, returns/refunds, settlement or other durable processes.

Test ID	Scenario and steps	Expected result	Priority
WF-001	Start a workflow with valid input.	Workflow starts with a unique business/workflow reference.	P1
WF-002	Attempt to start the same logical operation twice.	Workflow uniqueness/idempotency policy prevents duplicate business execution.	P0
WF-003	Stop the worker during an activity and restart it.	Workflow resumes or retries safely.	P0
WF-004	Cause a transient external-service failure.	Retry policy applies with appropriate limits and backoff.	P1
WF-005	Cause a permanent validation/business error.	Workflow fails or transitions to the documented error state without pointless retries.	P1
WF-006	Cause an activity to time out after its external side effect may have succeeded.	Activity reconciliation or idempotency prevents duplicate side effects.	P0
WF-007	Trigger a workflow cancellation.	Cancellation and compensation behavior follows the defined business contract.	P0
WF-008	Cause compensation to fail.	Failure is recorded and the operation remains visible for recovery; the system does not falsely report complete rollback.	P0

WF-009	Deliver duplicate external events to a workflow.	State transitions remain idempotent.	P0
WF-010	Deliver events out of order.	Events are validated against current state and invalid transitions are not applied.	P0
WF-011	Restart the Temporal service while workflows are waiting.	Workflow history persists and eligible workflows resume.	P0
WF-012	Inspect a failed workflow as an unauthorized user.	Sensitive workflow input and history remain protected.	P0
WF-013	Replay workflow history after a code change.	Workflow compatibility/versioning requirements are respected.	P1
WF-014	Run many workflows concurrently.	Worker limits and queue behavior remain within agreed resource limits.	P1
WF-015	Trigger a timeout-based reservation expiry.	Inventory is released only when payment/order state is safe to expire.	P0
WF-016	Verify workflow status exposed to the UI.	UI status corresponds to authoritative workflow/business state and refreshes correctly.	P1
WF-017	Kill a worker after a refund provider succeeds but before activity completion is recorded.	Recovery reconciles the refund rather than issuing another one.	P0

WF-018	Fail settlement processing halfway through.	Completed steps are recorded, unresolved steps are recoverable and duplicate payouts are prevented.	P0

Important: Do not write tests that expect Temporal alone to provide exactly-once payment or refund processing. Test idempotency at the Core/database/provider integration boundary.

11. Payments, billing, ledger and settlement integrity

Because this platform handles money for customers, vendors and the marketplace, these invariants deserve a dedicated financial test suite.

Test ID	Scenario and steps	Expected result	Priority
FIN-001	Calculate a customer order with no discounts or shipping fees.	Total matches the defined price calculation.	P0
FIN-002	Calculate a cart with different vendor commission rates.	Each vendor allocation uses the correct applicable rule.	P0
FIN-003	Apply discounts to a multi-vendor order.	Discount allocation follows the defined policy and sums to the total discount.	P0
FIN-004	Calculate tax and shipping for multiple order items.	Tax and shipping amounts follow configured jurisdiction/product rules.	P0
FIN-005	Issue a partial refund.	Refunded amounts and remaining refundable balance are correct.	P0

FIN-006	Attempt cumulative refunds exceeding the captured amount or eligible balance.	Core rejects the excess.	P0
FIN-007	Compare successful gateway charges with internal payment records.	Every charge reconciles or appears in an exception report.	P0
FIN-008	Compare internal refunds with gateway refund records.	Every refund reconciles or appears as an unresolved discrepancy.	P0
FIN-009	Compare vendor payable balances with ledger entries.	Ledger-derived balance equals the reported payable balance.	P0
FIN-010	Compare completed payouts with gateway/bank confirmations.	Payout status and reference match the provider result.	P0
FIN-011	Record a chargeback after a vendor payout.	Liability, vendor balance and platform adjustment follow the documented policy.	P0
FIN-012	Change a commission rate after an order is placed.	Existing orders retain their applicable historical commission basis unless an explicit adjustment is recorded.	P0
FIN-013	Recalculate a historical statement.	Original entries remain auditable; adjustments are represented explicitly.	P0
FIN-014	Attempt to edit or delete a posted financial ledger entry.	Direct mutation is prohibited or handled through an auditable reversal/adjustment.	P0

FIN-015	Generate a customer invoice and applicable credit note.	Documents reflect finalized financial data and follow numbering/retention policy.	P0
FIN-016	Reconcile platform revenue with commissions and other permitted revenue sources.	Revenue reporting follows the documented accounting definition.	P0
FIN-017	Process currency rounding for multiple items.	Rounding is deterministic and reconciles to the final amount.	P1
FIN-018	Attempt a payout to unverified or ineligible vendor payout details.	Payout is blocked pending required verification.	P0
FIN-019	Retry a payout after an ambiguous provider timeout.	Provider status is reconciled before another transfer is initiated.	P0
FIN-020	Verify settlement of orders that include returns, discounts, taxes and refunds.	Statement components and net payable reconcile.	P0

For these tests, keep money in integer minor units or an appropriate fixed-precision decimal representation. Never use binary floating-point arithmetic for authoritative currency calculations.

12. Logs, audit trail, monitoring and recovery tests

You specifically require proper logs. Do not treat a single application log table as sufficient: application logs, security audit records, business event history and Temporal workflow history serve different purposes.

Test ID	Scenario and steps	Expected result	Priority
LOG-001	Send a normal API request with a request/correlation ID.	Request logs contain the correlation ID and outcome.	P1
LOG-002	Send a request without a correlation ID.	The system generates one according to its tracing policy.	P2
LOG-003	Trigger a validation error.	Error is logged at an appropriate level without leaking sensitive input.	P1
LOG-004	Trigger an unexpected exception.	A diagnostic is recorded; the client receives a safe error response.	P0
LOG-005	Perform a sensitive admin action.	Audit record captures actor, action, target, time and outcome.	P0
LOG-006	Attempt a denied cross-tenant access.	Security event is recorded without logging credentials or excessive personal data.	P0
LOG-007	Complete a payment.	Payment, order and workflow references can be correlated across relevant logs.	P0
LOG-008	Receive a duplicate payment webhook.	Duplicate-event handling is traceable and does not create a duplicate charge.	P0
LOG-009	Run a failed refund workflow.	Failure reason, retry history and business references are available to authorized operators.	P0

LOG-010	Trigger a failed vendor payout.	Operational diagnostics and reconciliation references are available.	P0
LOG-011	Inspect logs as a vendor.	The vendor cannot read platform-wide logs or another vendor's sensitive diagnostics.	P0
LOG-012	Search logs by order or workflow reference.	Related events can be located without exposing unrelated tenants' records.	P1
LOG-013	Submit an access token or secret in a test request.	Secret redaction prevents the value from appearing in logs.	P0
LOG-014	Trigger a high rate of errors.	Alert thresholds work and the logging system remains usable.	P1
LOG-015	Fill or disconnect the log sink.	Application behavior follows a documented resilience policy; critical business transactions are not falsely reported as complete.	P1
LOG-016	Attempt to modify or delete an audit record through normal application APIs.	Audit history is protected from unauthorized mutation.	P0
LOG-017	Verify log retention and access controls.	Records are retained for the defined period and only authorized roles can access them.	P1
LOG-018	Trigger an alert for a stuck order or workflow.	Alert includes enough safe context to investigate and is routed to the appropriate operator.	P1

LOG-019	Trigger a notification failure.	Retry and failure history can be correlated with the triggering business event.	P2
LOG-020	Restore a test environment from backup.	Operational history and required financial/audit records are available according to the recovery plan.	P1

12.1 Required audit fields

For sensitive business events, verify fields such as:

- Event ID and timestamp.
- Actor ID and actor type.
- Tenant/organization and vendor context where applicable.
- Action and target resource.
- Outcome and safe reason code.
- Request/correlation ID and workflow ID where applicable.
- Before/after values for permitted changes, excluding secrets and sensitive data.

Do not log passwords, access tokens, private payment credentials, full card numbers or secret plugin configuration values.

13. UI usability, accessibility and browser compatibility

These tests apply to all five portals.

Test ID	Scenario and steps	Expected result	Priority
UI-001	Open every portal's primary page on a desktop browser.	Layout, navigation and content render without errors.	P1

UI-002	Open the same pages on a mobile viewport.	Responsive layout remains usable without essential controls being inaccessible.	P1
UI-003	Navigate using only the keyboard.	Interactive elements can be reached and operated in a logical order.	P1
UI-004	Use a screen reader on forms and primary navigation.	Labels, roles, validation and state changes are meaningfully announced.	P1
UI-005	Submit a form with invalid fields.	Errors identify the affected fields and explain how to correct them.	P1
UI-006	Submit a form successfully.	Duplicate submission is prevented while processing, and the resulting state is clear.	P1
UI-007	Trigger an API timeout.	Loading indicators stop appropriately and recovery/retry controls appear.	P1
UI-008	Load a table with no results.	Empty state is clear and does not resemble a loading failure.	P2
UI-009	Load a large table.	Pagination or virtualization works according to the design.	P2
UI-010	Refresh or open a deep link to a permitted route.	Route and session behavior are correct.	P1
UI-011	Open an unauthorized deep link.	UI and Core deny access.	P0

UI-012	Use browser Back after checkout or a sensitive action.	Stale screens do not enable duplicate purchases or actions.	P0
UI-013	Test long names, large values, Unicode and translated text if supported.	Content remains readable and controls remain usable.	P2
UI-014	Test focus, contrast, text scaling and visible validation.	Essential interactions remain accessible.	P1
UI-015	Test in the supported browser matrix.	Critical workflows behave consistently across supported browsers.	P1
UI-016	Open the UI while Core is unavailable.	Error and retry states are clear; no false success is displayed.	P1
UI-017	Verify navigation for a user with a restricted role.	Only authorized actions are shown, while Core remains the authoritative enforcement point.	P0
UI-018	Verify confirmation for destructive actions.	High-impact operations require the appropriate confirmation or approval process.	P1

14. Performance, reliability and deployment tests

Test ID	Scenario and steps	Expected result	Priority

OPS-001	Load a representative product catalog under expected traffic.	Response times meet agreed performance targets.	P1
OPS-002	Run concurrent checkouts against limited inventory.	No overselling or duplicate financial side effects occur.	P0
OPS-003	Run concurrent requests against the same cart.	Final cart state is consistent according to the update policy.	P1
OPS-004	Run concurrent webhook processing.	Idempotency and database constraints preserve correctness.	P0
OPS-005	Simulate database connection exhaustion.	Requests fail safely, recover when connections return, and do not leave false business success states.	P0
OPS-006	Simulate Temporal worker downtime.	Durable workflows remain recoverable.	P0
OPS-007	Simulate payment-provider downtime.	New payments fail or remain pending safely; recovery follows policy.	P0
OPS-008	Simulate email/SMS provider downtime.	Commerce operations remain consistent and notifications retry appropriately.	P1
OPS-009	Deploy a backward-compatible database migration.	Existing functionality remains available according to the rollout strategy.	P1
OPS-010	Roll out a new Core version while workflows are running.	Workflow compatibility and deployment procedures prevent corruption.	P0

OPS-011	Roll out a new UI version.	Routes, authentication and API contracts remain compatible.	P1
OPS-012	Build and run the Docker images in a clean environment.	Required services start with correct configuration and health checks.	P1
OPS-013	Verify secrets and environment-specific URLs in deployment configuration.	No production secrets are committed to source or embedded in client-side bundles.	P0
OPS-014	Run the automated test suite in CI.	Required tests run and failures block the relevant release workflow.	P1
OPS-015	Verify backup and restore procedures.	The environment meets its defined recovery point and recovery time objectives.	P1
OPS-016	Run a dependency and container vulnerability scan.	Critical findings are addressed or formally risk-accepted before release.	P1
OPS-017	Test graceful service shutdown during active requests.	Operations finish or fail safely; partial business updates are not silently committed.	P1
OPS-018	Run a sustained workload and inspect database, worker and API resource usage.	Resource consumption stays within agreed limits or triggers actionable alerts.	P1

Performance targets should be explicitly defined for your expected traffic, such as product-search latency, checkout latency, webhook processing delay and dashboard response time. Do not use arbitrary thresholds as release criteria without considering your deployment environment.

15. Test cases for the additional operational experiences

These are not separate portals in your initial design. They are additional workflows that should be covered either in Platform Admin, Customer Account or Vendor Portal, depending on your permissions model.

Test ID	Scenario	Expected result	Priority
EXT-001	Support agent opens an assigned ticket.	Only tickets within the agent's permitted scope are visible.	P1
EXT-002	Support agent escalates a payment dispute.	Escalation is recorded and routed to an authorized finance/admin role.	P1
EXT-003	Finance operator approves a payout.	Approval and payout initiation are separately recorded if dual approval is required.	P0
EXT-004	A delivery partner updates a shipment.	Only assigned shipments can be changed, and the update is auditable.	P1
EXT-005	A plugin developer submits a new plugin version, if that feature is introduced.	Publication requires the defined validation, review and approval process.	P1
EXT-006	An organization requests offboarding.	New activity, outstanding orders, vendor balances, data retention and deletion requirements are handled according to policy.	P0

EXT-007	A customer requests account/data deletion.	Applicable privacy, retention and financial-record obligations are respected.	P0
EXT-008	A vendor requests account closure with outstanding settlements.	Closure is blocked or follows an explicit settlement and offboarding process.	P0

16. How to implement this test suite in your repositories

For your two repositories, I would organize automated tests as follows.

Core — FastAPI

Unit tests for business rules; API integration tests for routes, permissions and schemas; database integration tests for transactions, constraints and migrations; Temporal tests for retries and recovery.

UI — Next.js

Component tests, form validation, role-based navigation, loading/error states, and end-to-end browser tests for the five portal experiences.

Temporal and integrations

Workflow tests with simulated payment, shipping, notification and payout providers, plus controlled failure injection and recovery tests.

GitHub Actions / CI

Run unit and integration tests, migration checks, security checks and build validation before publishing deployable images. Run browser end-to-end tests in an appropriate test environment.

Use dedicated test fixtures to create users, organizations, vendors, products and orders. Avoid hardcoding production IDs or depending on the state left behind by a previous test.

17. Recommended release gates

Before allowing real customers to use the marketplace, enforce these gates:

- Gate A — Access control: all P0 authentication, authorization and tenant-isolation tests pass.
- Gate B — Shopping: catalog, wishlist, cart, pricing and checkout tests pass.
- Gate C — Money and inventory: payment idempotency, inventory reservation, refunds and ledger reconciliation tests pass.
- Gate D — Fulfillment: cancellation, shipment, delivery, returns and digital fulfillment tests pass for every product type you actually offer.
- Gate E — Operations: admin/vendor workflows, audit records, failure recovery, backups and deployment checks pass.

Do not consider a portal complete just because all its pages render. Its backend authorization, business transitions, database invariants and failure paths must also pass.

18. Track execution in a test management sheet

For this many cases, a spreadsheet will be much easier to maintain than a long document. Use one row per test case with these columns:

Field	Purpose
Test ID	Stable identifier, e.g.
Portal / module	Platform Admin, Tenant Admin, Vendor, Plugin or Customer
Priority	P0–P3
Preconditions	Required users, data and state
Steps	Reproducible actions
Expected result	Business and technical acceptance criteria
Actual result	What happened during execution

Field	Purpose
Status	Not Run, Pass, Fail, Blocked, Not Applicable
Defect ID	Link to GitHub issue or pull request
Automation	Manual, API, integration or browser E2E
Evidence	Sanitized logs, screenshots or report references

The tables above provide the initial test catalogue. For genuinely exhaustive coverage, each scenario should eventually have explicit test data, boundary values, state-transition variants, permission variants and recovery cases.

My recommendation for your next implementation step: establish the P0 test cases first—particularly cross-tenant authorization, inventory concurrency, duplicate checkout, payment reconciliation, partial refunds and vendor settlement. Those tests protect the foundations of the marketplace before you expand the UI or onboard more vendors.